

TM261 – Agent-Led Test Management: Hands-on Lab with Joule in SAP Cloud ALM

Date: Thursday, 29 October 2026 · SAP TechEd 2026, Berlin
Presenters: Christina Pickhardt · Niclas Weidner · Tobias Keller · Daniel Lorenz
Scenario: Make-to-Stock Production – Discrete Manufacturing (BJ5)

Agenda

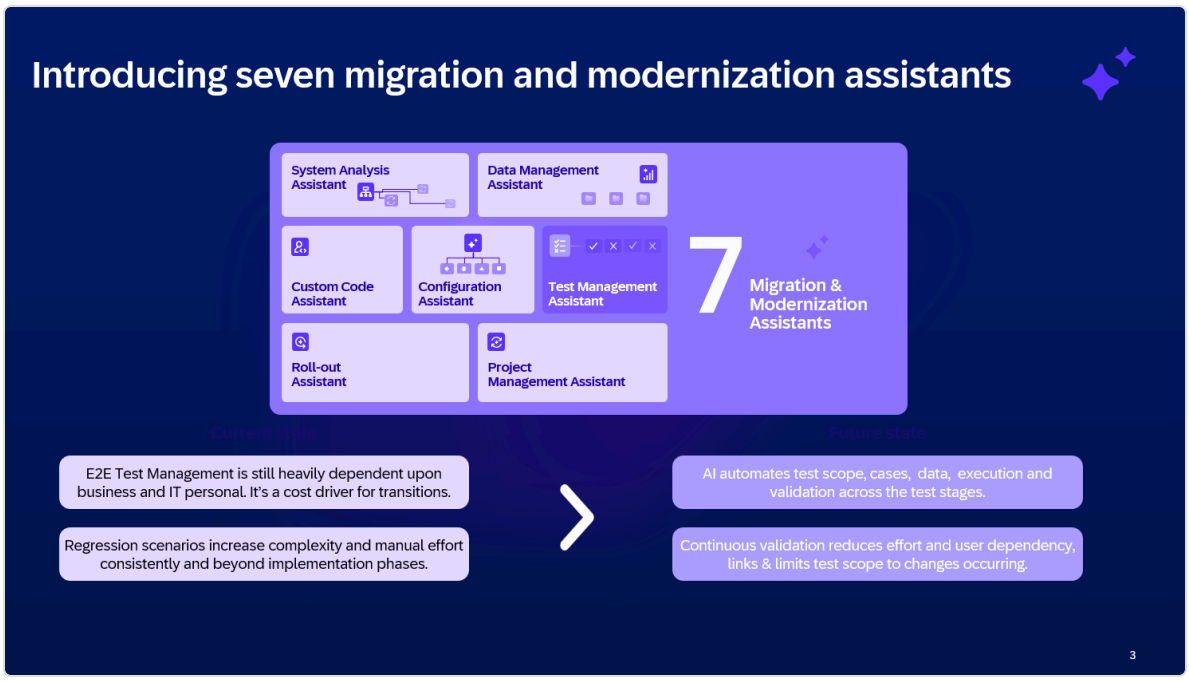
#	Section	Topic
–	Introduction	Test Management Assistant: context and agent overview
1	Getting Started	Log In & Orient Yourself
2	Generate *Requirements*	Turn an interview transcript into *Requirements* with AI
3	Scope with AI	Scoping agent; compare with manual scoping
4	Plan with AI	Planning agent; HITL Gate 2; review
5	Generate *Test Cases*	TC generation agent; quality review; parameters
6	Generate Test Data	Data agent; classify & explore; dependencies
7	Recap & Outlook	Time saved; roadmap; key takeaways

Introduction: Test Management Assistant

Context – SAP Migration & Modernization Assistants

Before we jump into the hands-on, a quick word on context, because what you're about to use is not an isolated feature.

SAP has developed seven AI-powered assistants for Migration and Modernization scenarios, built to help customers move from SAP ECC to S/4HANA Cloud Private Edition faster and with significantly less manual effort:



01 – Seven SAP Migration & Modernization Assistants

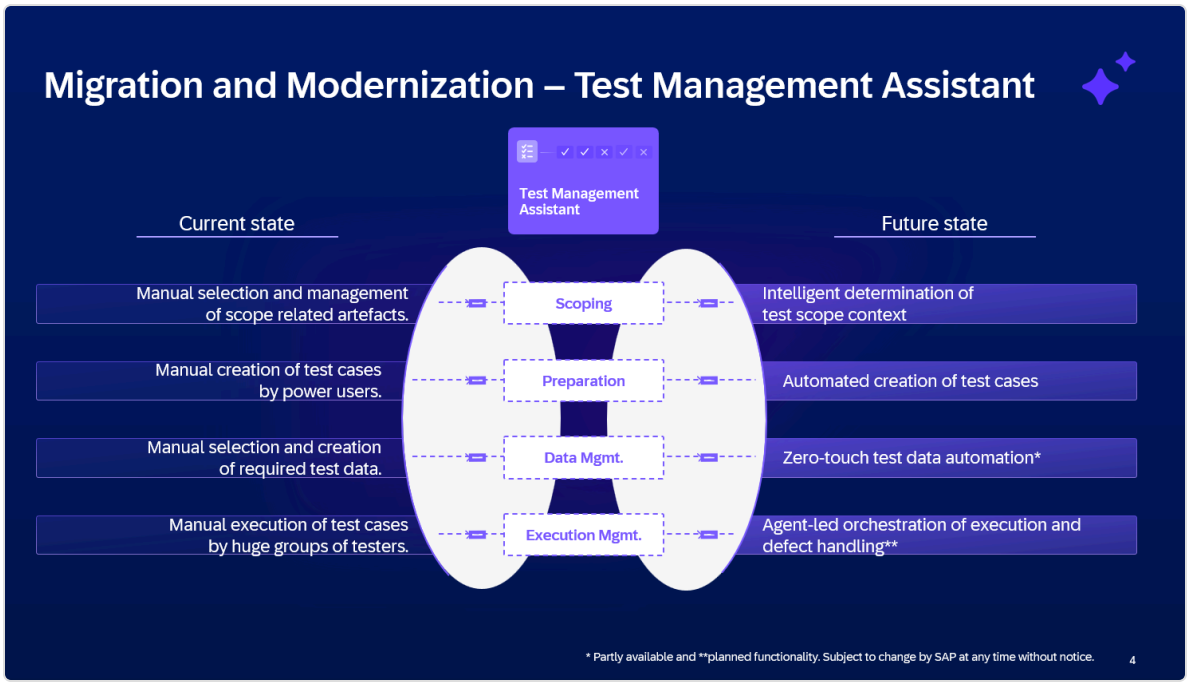
The **Test Management Assistant** is one of these seven, and Test Management is an area where the shift from manual to AI-driven work matters most:

Current State	Future State
E2E Test Management is still heavily dependent upon business and IT personnel; it's a cost driver for transitions.	AI automates test scope, cases, data, execution and validation across the test stages.
Regression scenarios increase complexity and manual effort consistently and beyond implementation phases.	Continuous validation reduces effort and user dependency, links & limits test scope to changes occurring.

The TMA targets the upstream side of this: the preparation of manual end-to-end testing. It sits at the quality gate of every migration, and as you'll see today, it's where AI can compress days of manual work into minutes of guided review.

What is the Test Management Assistant?

The Test Management Assistant is not a single AI feature; it's a pipeline of four specialized agents, each transforming a stage of manual Test Preparation into an AI-driven capability:



02 – Test Management Assistant: four-agent pipeline

> **Today's hands-on covers the first three agents**: Scoping, Preparation, and Data Management. Execution Management is on the roadmap.
>
> ** Partly available and **planned functionality. Subject to change by SAP at any time without notice.*

Continue to Chapter 1 – Getting Started

← Back to Introduction

Chapter 1 – Getting Started

Your Story Today

You are a **Test Business Process Expert** at a company rolling out SAP S/4HANA. As part of that rollout, your team activated SAP S/4HANA *Scope Item* BJ5: Make-to-Stock Production – Discrete Manufacturing, and adapted it to fit your specific operational needs; creating a custom process variant called **BJ5_simple**. Your task today: **prepare test coverage for this process, with a focus on validating the role assignments and workflow changes** that came with the customisation. You're starting from scratch.

In this hands-on you'll work with AI agents in SAP Cloud ALM Test Management to speed up every stage, from *Requirements* to test data, dramatically reducing the manual effort that would normally take days.

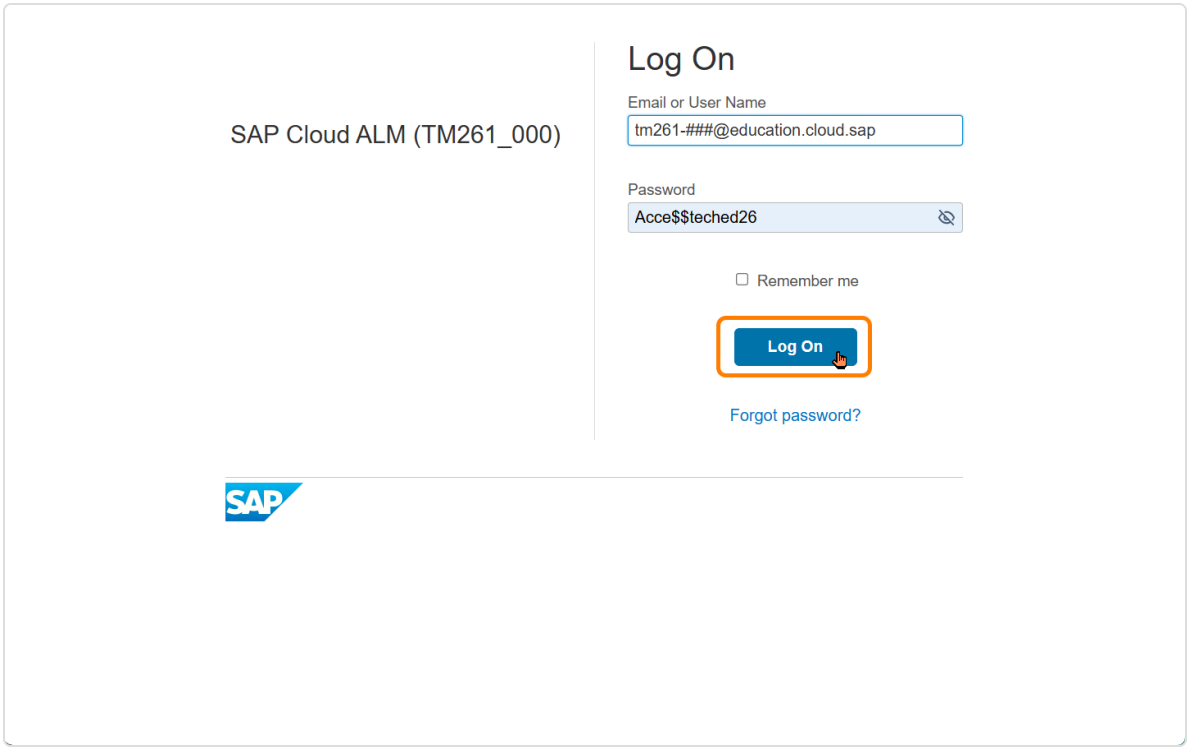
1.1 – Log In to SAP Cloud ALM

It's the start of your work day, and preparing test coverage for the newly customised process is on your agenda. SAP Cloud ALM Test Management is where this work happens: the platform where you plan, execute, and track test projects end to end, with every test artifact traceable back to *Requirements*, providing documented assurance of what has been validated and why.

Open your browser and navigate to the Cloud ALM tenant.

> **Your credentials:**
>
> | Field | Value |
> |---|---|
> | **Tenant** | SAP Cloud ALM (TM261) |
> | **User** | `tm261-###@education.cloud.sap` |
> | **Password** | `Acce$$teched26` |
>
> Replace **###** with your workstation number, zero-padded to three digits: desk 3 → `003`, desk 18 → `018`, desk 42 → `042`. Your number is on the tent card on your desk.
>
> Whenever you see `###` in a field value, filename, or prompt in this guide, that's your cue to substitute your own number. Screenshots in this guide use `099` as the example; wherever you see `099`, read it as your own number.

1. Enter your username and password, then click **Log On**.



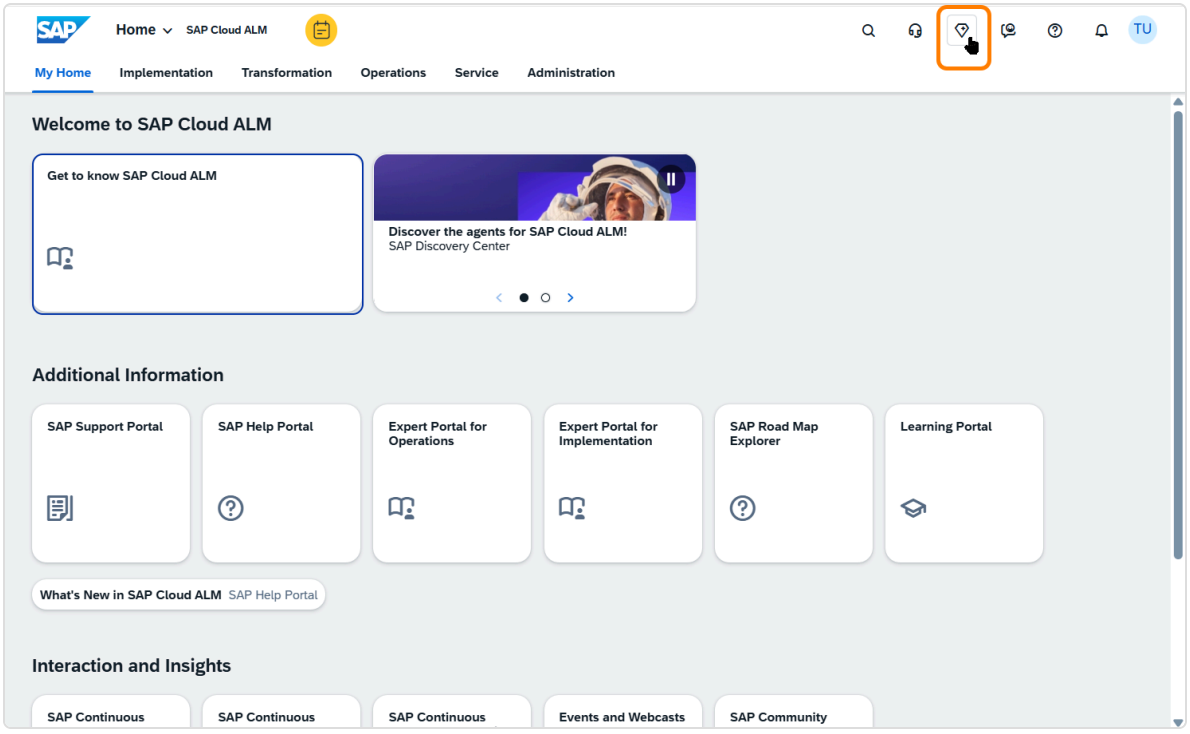
03 – Login to SAP Cloud ALM

1.2 – Open Joule

After logging in you land on the SAP Cloud ALM home page. We'll explore more of the platform in a moment, but first: let's call your most valuable tool for today: **Joule**.

Joule is a chat-style AI interface built into SAP Cloud ALM. You'll use it throughout this session to interact with AI agents, ask questions, and drive the Test Preparation workflow; all in natural language.

1. Look for the **diamond icon** in the upper-right area of the screen and click it to open Joule.

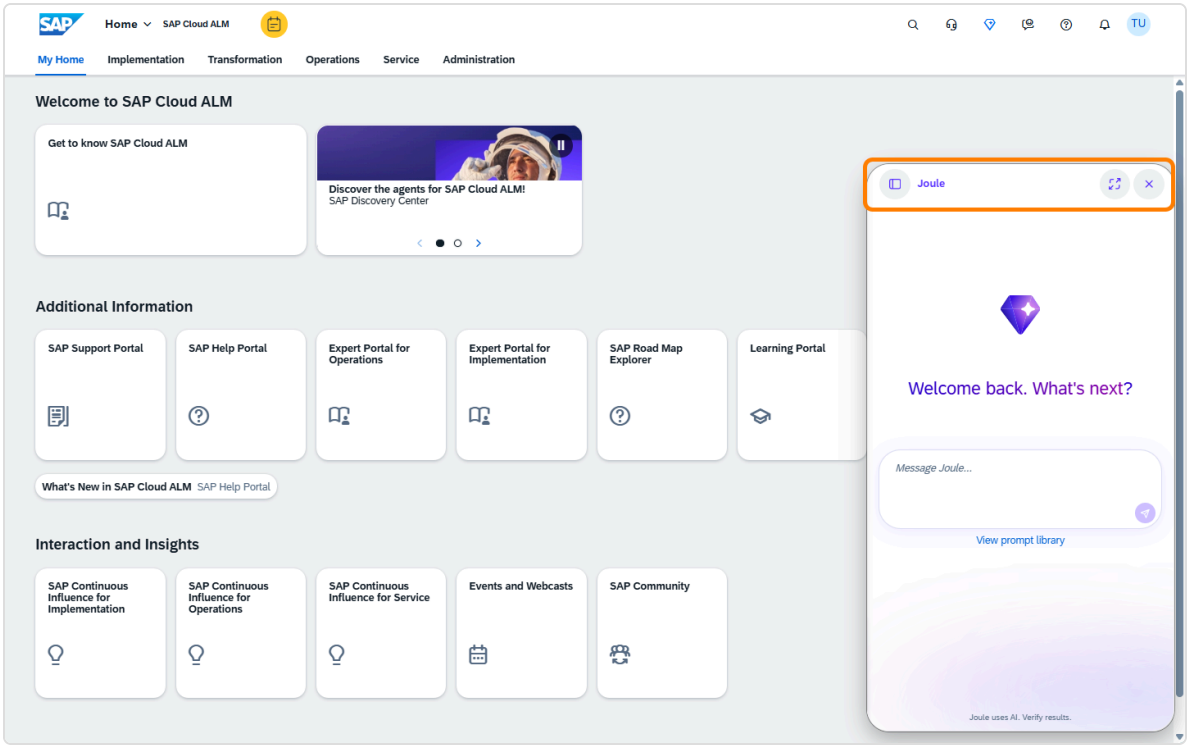


04 – SAP Cloud ALM home page

2. Joule opens as a panel that slides over the application screen; it's always available, regardless of where you are in SAP Cloud ALM.

The panel header gives you three controls:
- **Expand icon**: open Joule in full-screen mode
- **Close (×)**: close the panel
- **Sidebar icon**: manage multiple concurrent conversations (you can run several Joule chats in parallel, one per topic or agent)

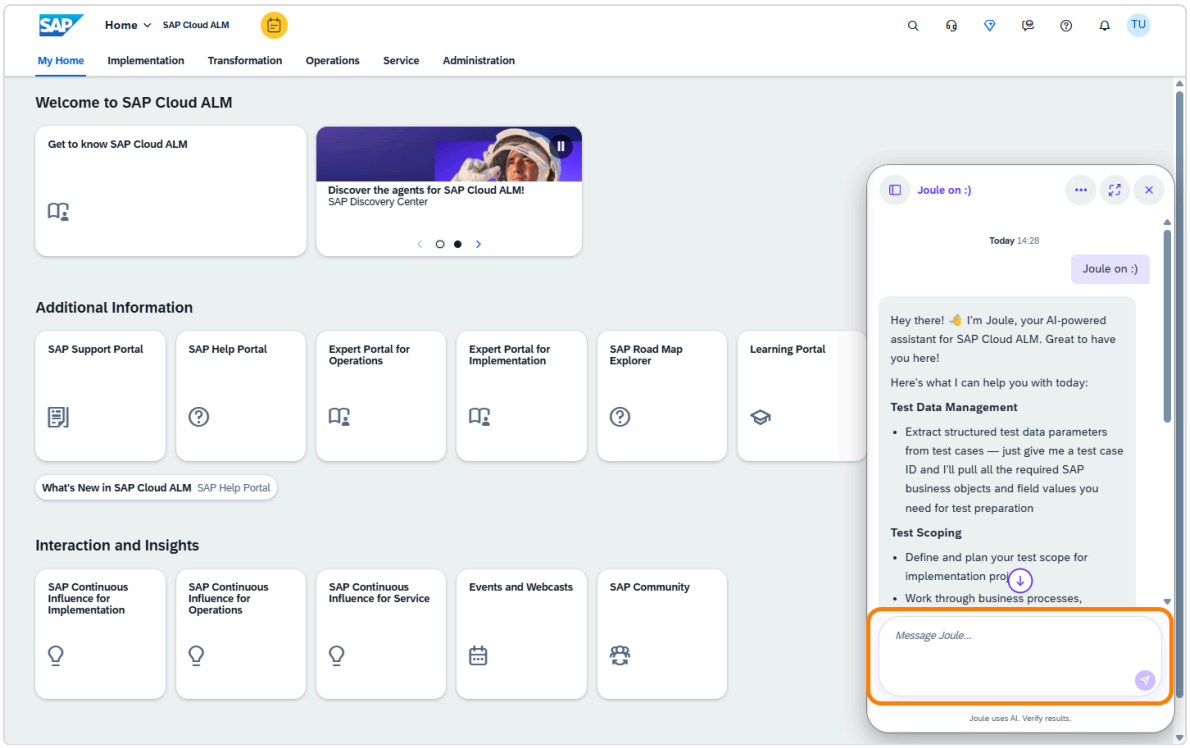
> **Good to know:** Joule is stateful. Close the panel, navigate elsewhere, reopen it; your conversation is still there where you left it.
> The sidebar shows your conversation list: up to **10 conversations can be active** at the same time, with older ones moved to **Expired**.



05 – Joule panel open

3. Wake up Joule with a simple message in the input box at the bottom; just type naturally, for example: **"Joule on :)"** or any greeting.

Joule will start processing; this can take a moment, feel free to read ahead while waiting. When it responds, you'll see an introduction of its capabilities covering areas like **Test Data Management**, **Test Scoping**, **Test Planning**, and **ALM Project Setup**. These are exactly the capabilities you'll use today.



06 – Joule greeting and capabilities

> ⚠ **Before you continue: a word on stability.** The agentic capabilities you're about to use involve complex multi-step orchestration across multiple AI skills and agent calls. This session runs on a shared productive tenant, designed for normal corporate use, not for dozens of simultaneous participants. Expect some variability in response times.
>
> If Joule appears frozen, read ahead and give it a moment. If it seems to have timed out or lost context entirely, send a gentle follow-up ("still there?" or "hi") to wake it up, or re-send the original prompt. This is a known behaviour in the agentic framework; don't be surprised if it happens.

1.3 – Jump Ahead: What Happens Without Preparation? *(Optional)*

> **Optional step**: try this if you want to see what happens when you skip straight to *Test Case* generation without proper setup.

In the initial view you'll notice the process has a BPMN flow diagram, but nothing else linked to it. Let's see what Joule does when asked to work with that incomplete input.

In the Joule message window, prompt it to generate *Test Cases* directly; specifying the process and project:

> *"I need to test process "Beta_Straight Make-to-Stock Production – Discrete Manufacturing (BJ5_simple)" in project "###_TestProject""*
>
> Replace **###** with your workstation number.
>
> **Tip:** Click the **expand icon** (⤢) in the Joule panel header to open Joule in full-screen mode; much easier to read the agent outputs, especially the longer ones coming later in this session.

Joule navigates to the process in your project and analyses its content, but it hits a problem:

> ⚠️ **Blocker: No Requirement or User Story Assigned**
>
> Joule found the process and confirmed the BPMN diagram is available, but it immediately flags a blocker: **no Requirement or User Story is linked to the process**.

> The agents require at least a *Requirements* description before *Test Cases* can be generated; the more background they have, the better the output quality.
>
> **Garbage in, garbage out** is true for AI agents too. The quality of the output depends directly on the quality of the input. Before Joule can do useful work, we need to enrich the process with *Requirements* and *User Stories*. That's exactly what we'll do next.

Chapter 2 – Feed the Agents

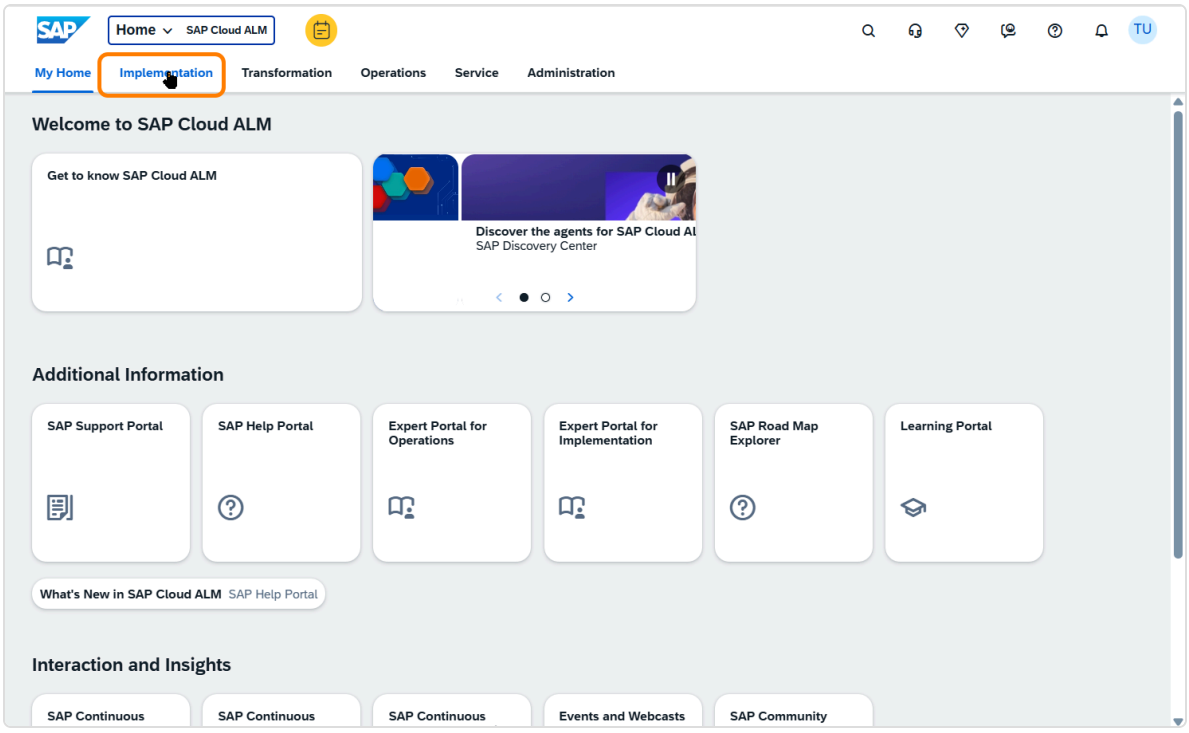
Before the AI agents can do useful work, the process needs to be populated with the documentation they'll work from. A BPMN diagram is already there, as you'll see; but a *Requirements* document and linked *User Stories* add the business context and process intent that language models are best at processing.

In this chapter you'll navigate to your project, assign pre-prepared *User Stories*, and generate a *Requirements* document from an expert interview transcript.

2.1 – Navigate to Your Project

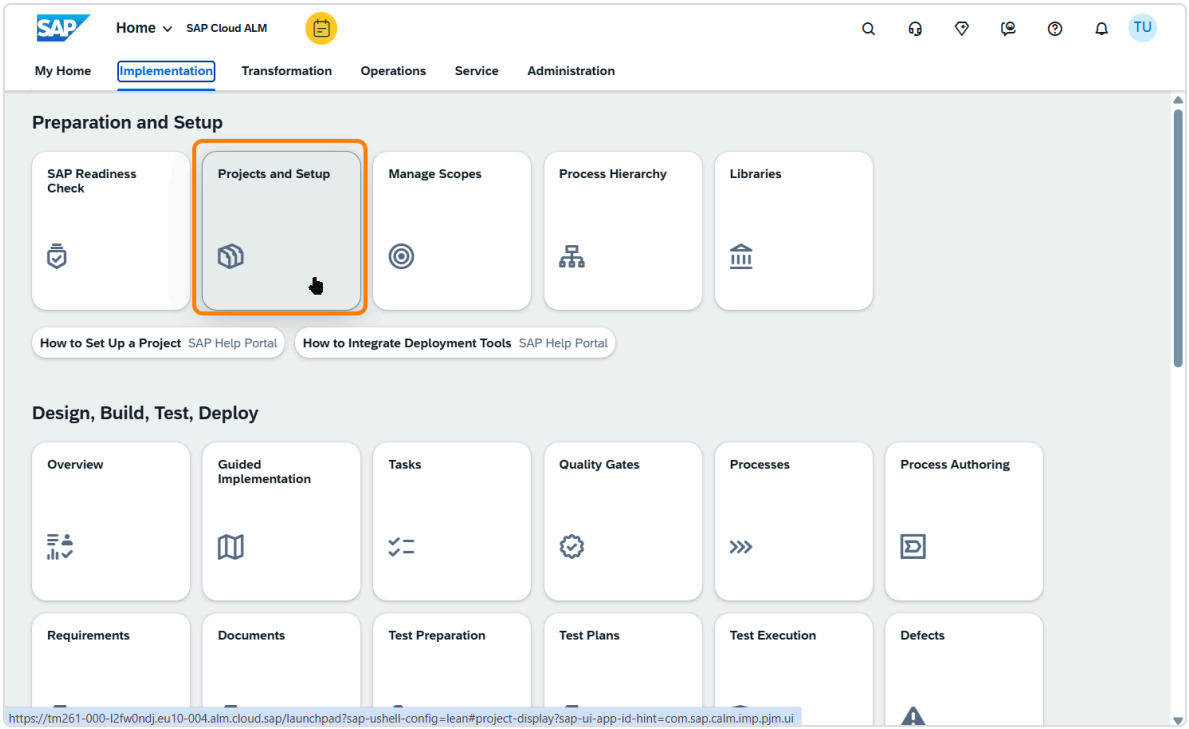
Time to find your process. Give Joule a break first; close the panel (click × in the panel header), then navigate to the project.

1. Back on the Cloud ALM home screen, click the **Implementation** tab in the top navigation bar. This is the launchpad for all Test Management related apps and more: *Requirements*, process authoring, Test Preparation, *Test Plans*, Test Execution, *Defects*.



07 – Implementation tab

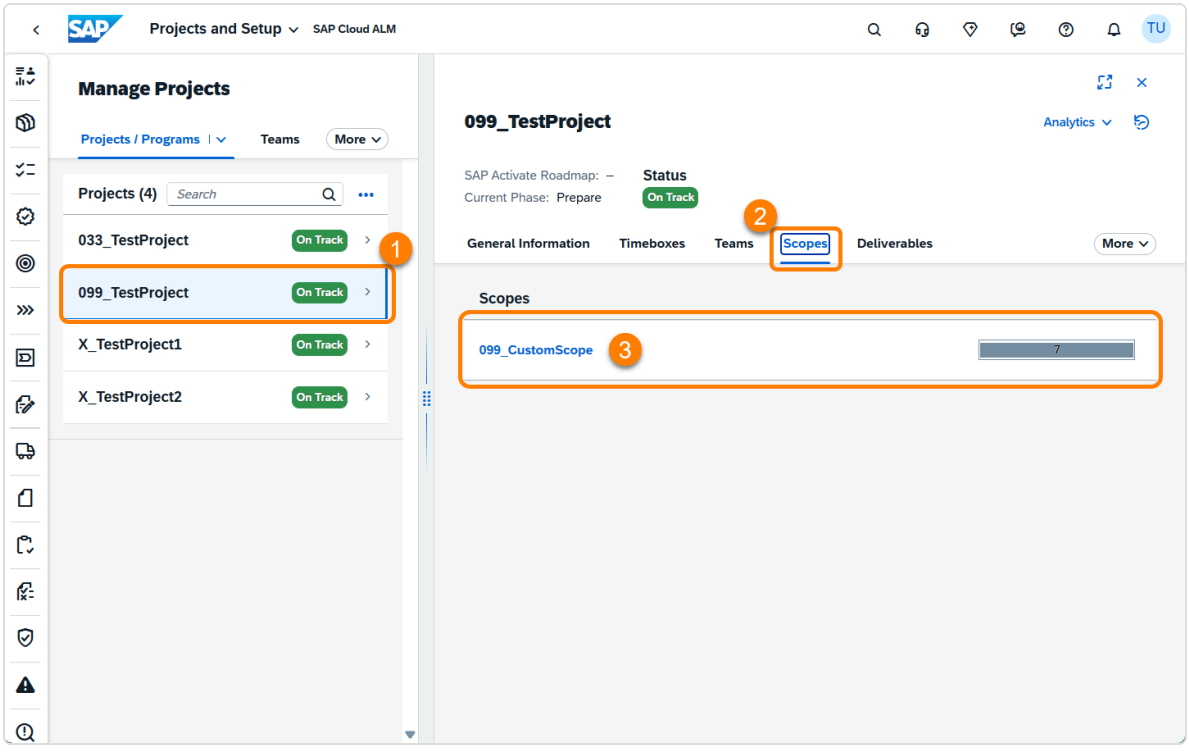
2. In the **Preparation and Setup** section, click the **Projects and Setup** tile to reach the overview of all projects in Cloud ALM.



08 – Projects and Setup tile

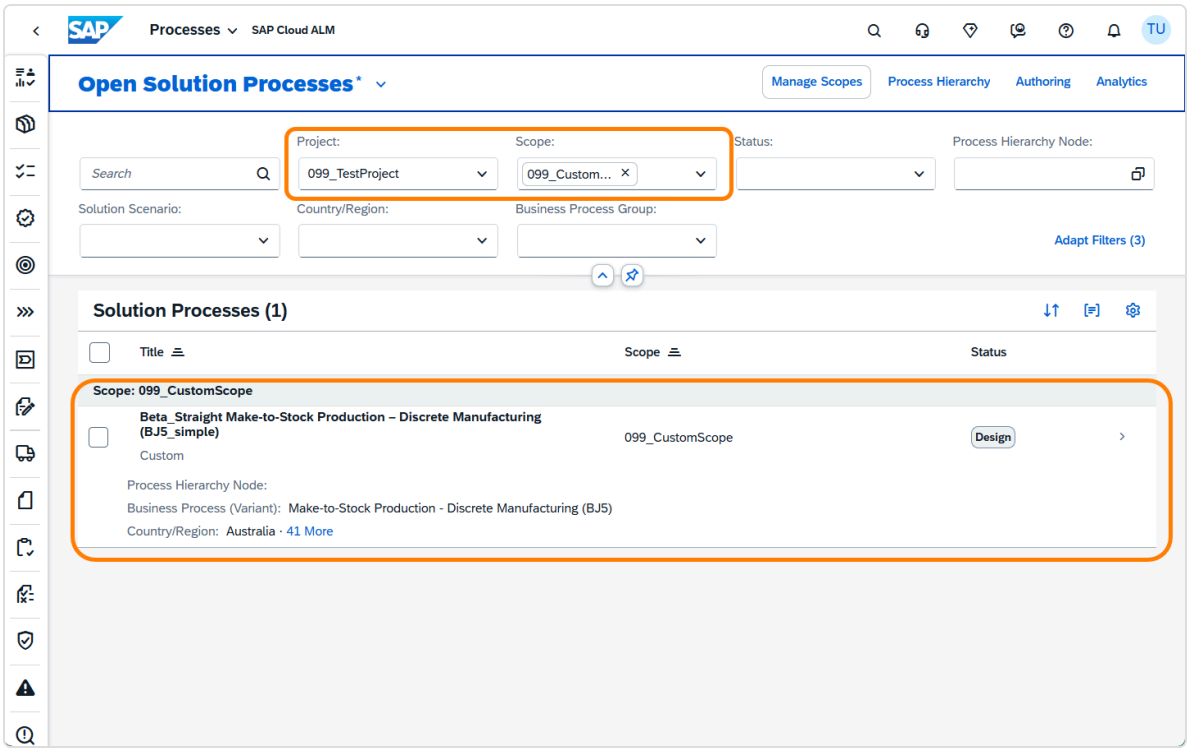
3. In the **Manage Projects** list, locate your project: `###_TestProject` (replace ### with your workstation number, e.g. `033`). Click it to open the project details.

4. In the project detail view, click the **Scopes** tab. This section can hold multiple scopes; here you'll find exactly one: `###_CustomScope`, the container for all the BJ5_simple process content you'll work with today. Click on it.



09 – Project list and scope

5. The view switches to **Open Solution Processes**, with filters pre-set to your project and scope. In a real-life implementation this list could contain dozens of processes; today you have exactly one: `Beta_Straight Make-to-Stock Production – Discrete Manufacturing (BJ5_simple)`. Click on it to open its details.

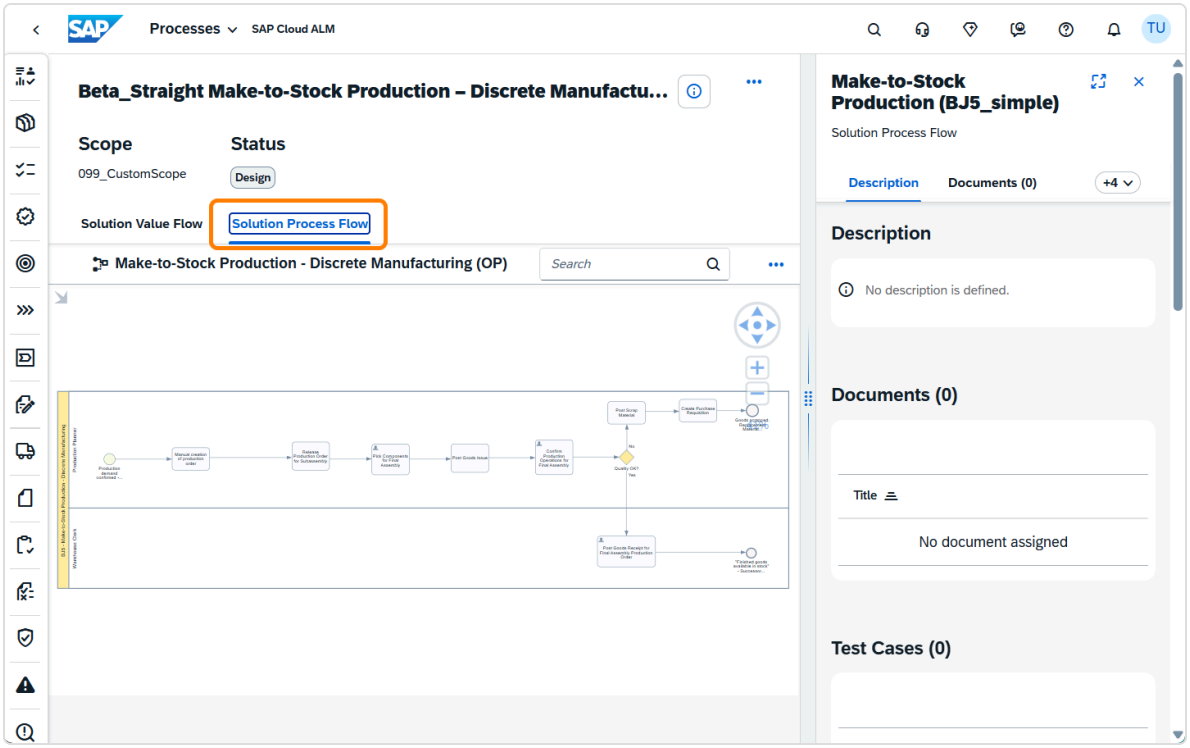


10 – BJ5 process in the custom scope

6. The process detail view opens. A process record in Cloud ALM can hold a lot of information; for now, not much has been added here.

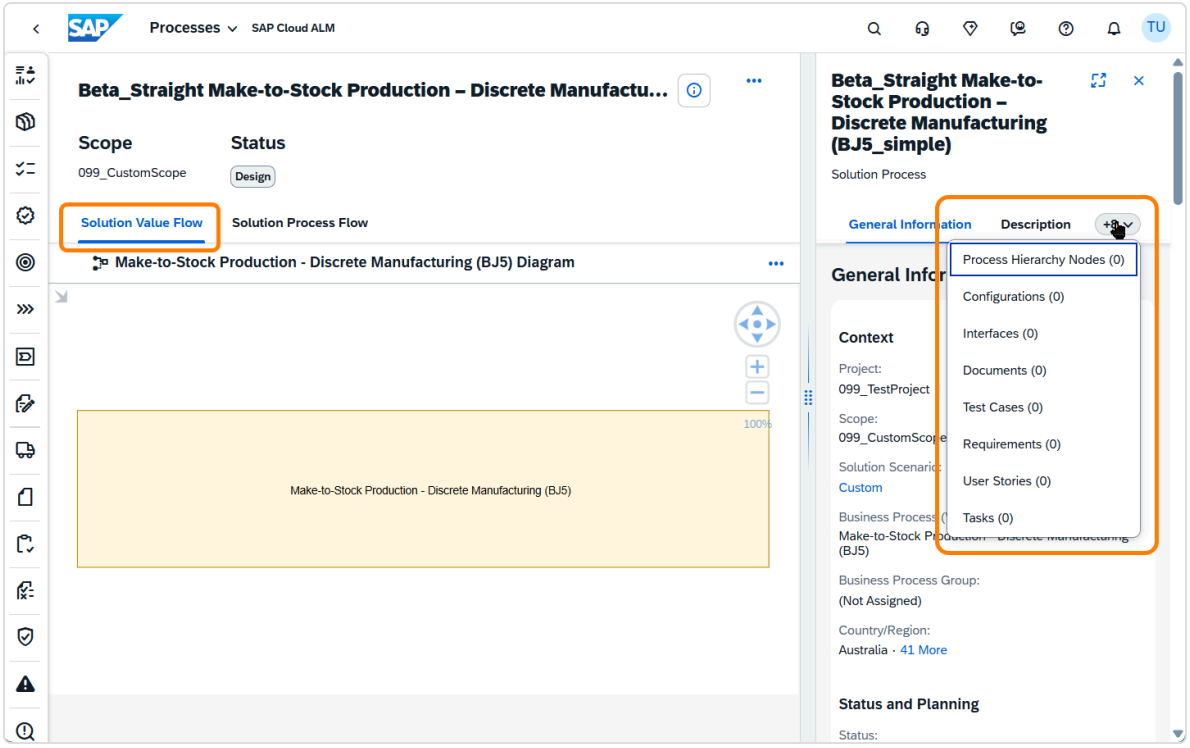
7. Click the **Solution Process Flow** tab (next to Solution Value Flow). Here you'll find something promising: a BPMN process diagram showing the BJ5 flow across two swimlanes; Production Planner and Warehouse Clerk. This diagram was imported from a process modelling tool such as **SAP Signavio** and is stored natively in Cloud ALM (a direct live integration with Signavio is planned as a future capability).

> **But here's the catch:** the diagram alone is not sufficient for the AI agents to generate high-quality test output. It provides the process skeleton, but without *Requirements* and *User Stories* the agents can't derive meaningful test coverage from structure alone.



11 – BJ5 process flow diagram

8. Return to the **Solution Value Flow** tab. On the right-hand panel, click **+8** to expand the full list of content types that can be linked to this process. All counters are at zero: no *Requirements*, no *User Stories*, no *Test Cases*.



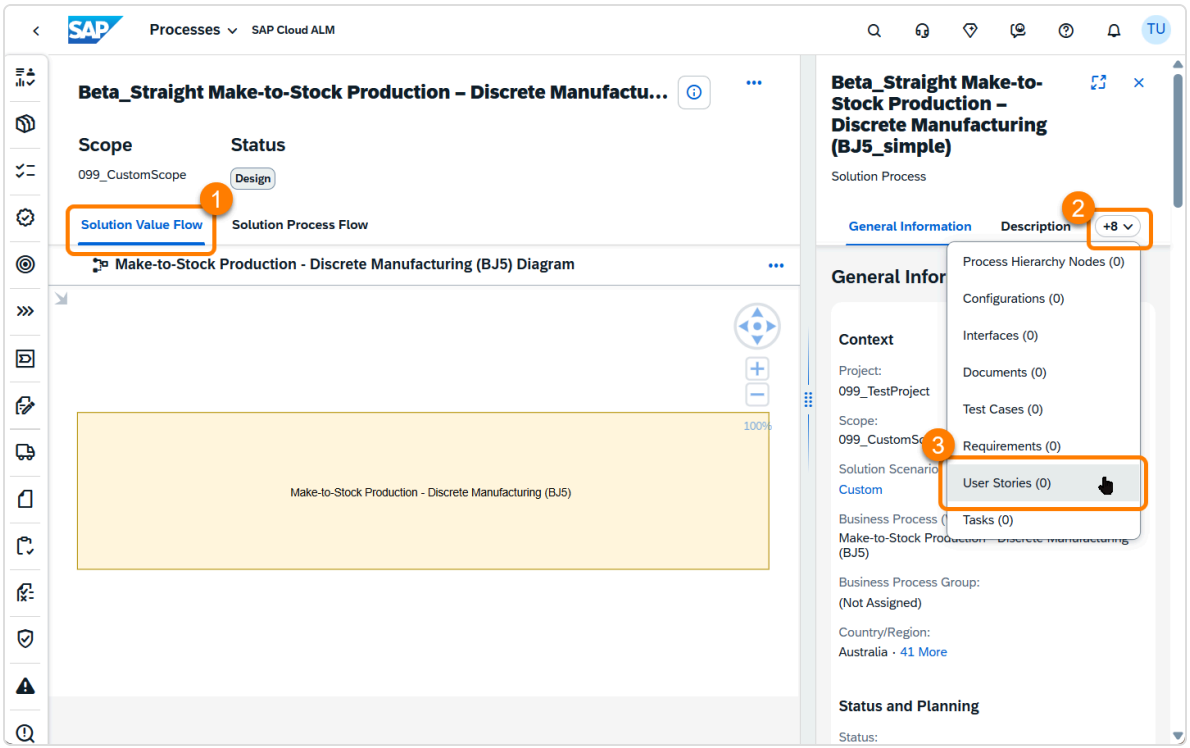
12 – Process detail: all content at zero

> **This is the root cause** of Joule's rejection in Step 1.3. Without *Requirements* or *User Stories*, the agents have no process intent to work from; and by design, they won't make things up. Before we can generate anything meaningful, we need to populate this process, and that's exactly what the next steps will do.

2.2 – Assign User Stories

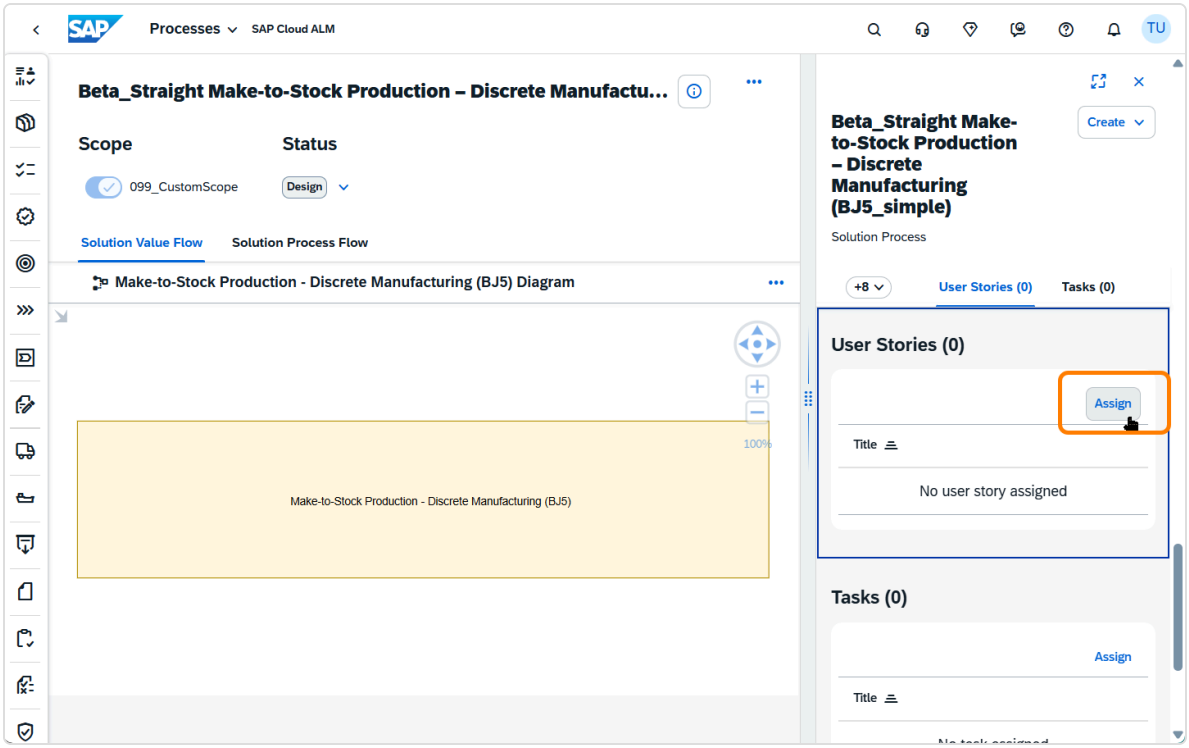
User Stories for the BJ5_simple process have already been defined and created in the system; they just haven't been linked to this process yet. Let's do that now.

1. Make sure you are on the **Solution Value Flow** tab. On the right-hand panel, click the **+8** dropdown and select **User Stories** to bring that section into view.



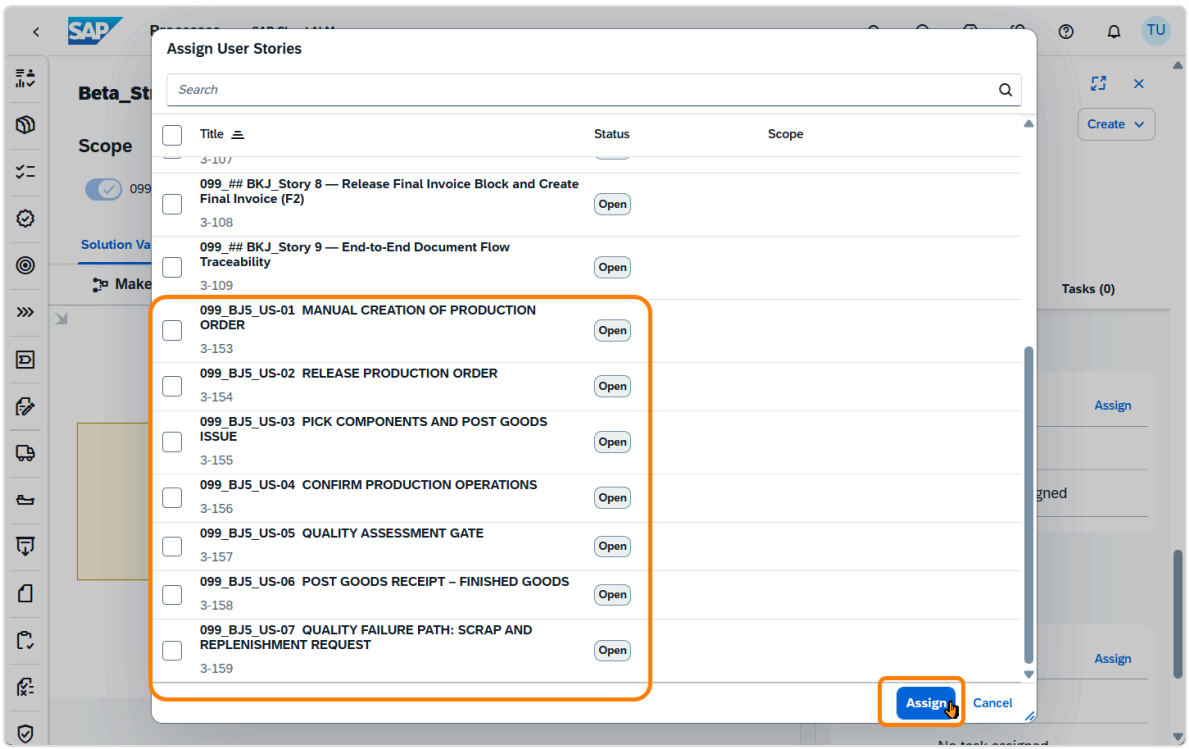
13 – Opening User Stories section

2. The **User Stories** section appears in the right panel, currently empty ("No *User Story* assigned"). Click the **Assign** button to open the selection dialog.



14 – Assign User Stories dialog

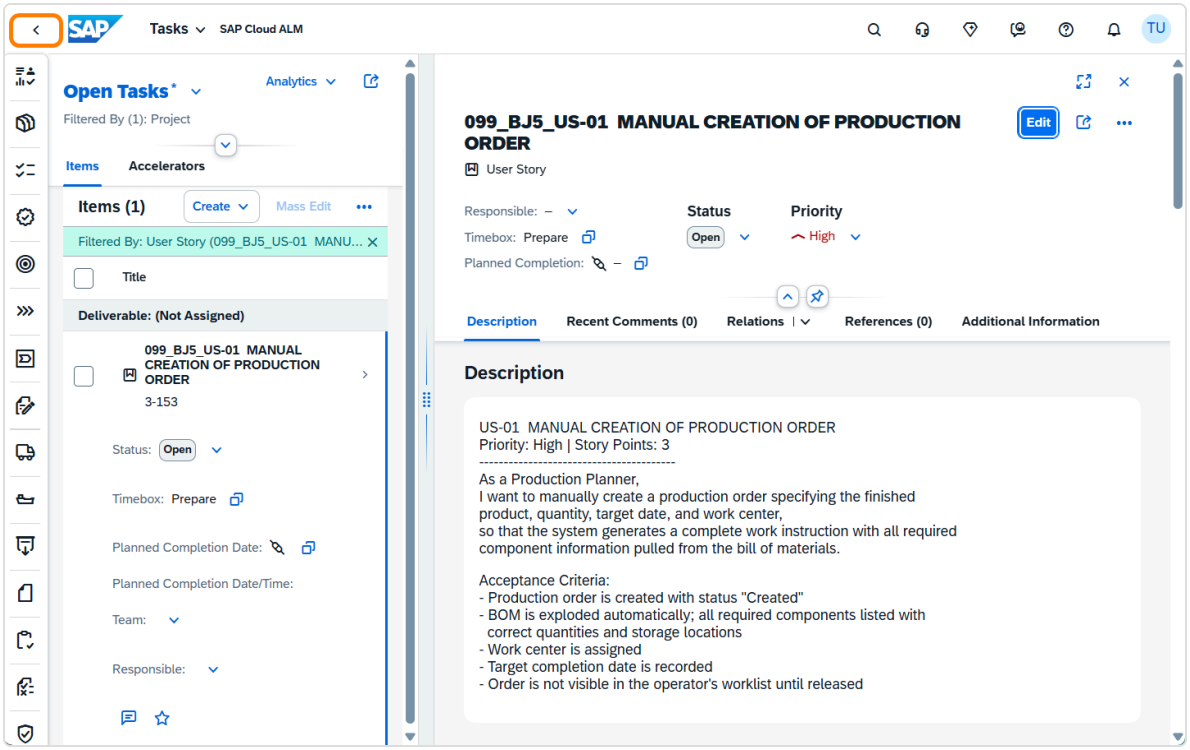
3. The **Assign User Stories** dialog opens, pre-populated with *User Stories* prepared for this process. Find the 7 entries prefixed **BJ5_US-**, select all of them, then click **Assign**.



15 – Selecting all 7 BJ5 user stories

4. Back in the process, open one of the *User Stories* to inspect it. Each story follows the same structure: **story points**, a **user story** narrative ("As a [role], I want to... so that..."), and

granular **acceptance criteria**: exactly the kind of structured input the AI agents need.



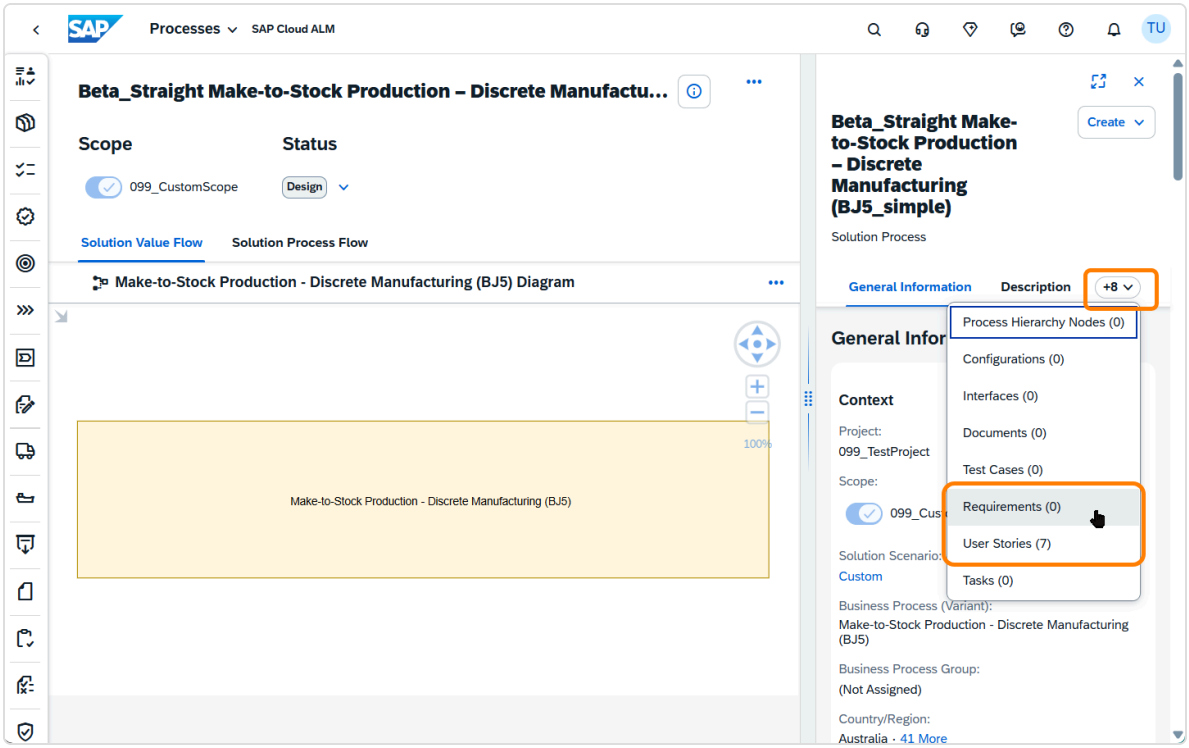
16 – User story: narrative and acceptance criteria

When done, click the **back arrow** (←) at the top left to return to the process view.

5. Back in the process, open the **+8** dropdown again; **User Stories** now shows **(7)**. The assignment worked.

But look at **Requirements (0)**: still empty, and arguably the most important input for the agents. A well-formed *Requirements* document gives the agents the building blocks of the process: business rules, role responsibilities, preconditions, expected outcomes, and the constraints that separate a happy path from an error scenario. This is the context that lets them generate meaningful, traceable test coverage rather than generic steps.

Here's the good news: you were recently in a structured interview with a BJ5 process expert, walking through the process step by step. Your colleague promised to post the transcript in SAP Cloud ALM. Time to retrieve it and turn it into a *Requirements* document.



17 – Requirements count still zero

Summary

Chapters 1 and 2 (so far):
- [x] Logged in and opened Joule
- [x] BJ5_simple process located in `###_TestProject / ###_CustomScope`
- [x] 7 *User Stories* assigned to the process
- [ ] *Requirements* document: still missing; covered in step 2.3

Continue to Chapter 2 – Feed the Agents (continued)

2.3 – Generate Requirements from an Interview Transcript

Prerequisite: You have completed Chapter 2 – steps 2.1 and 2.2; the BJ5 process is set up with *User Stories*.

What You'll Do

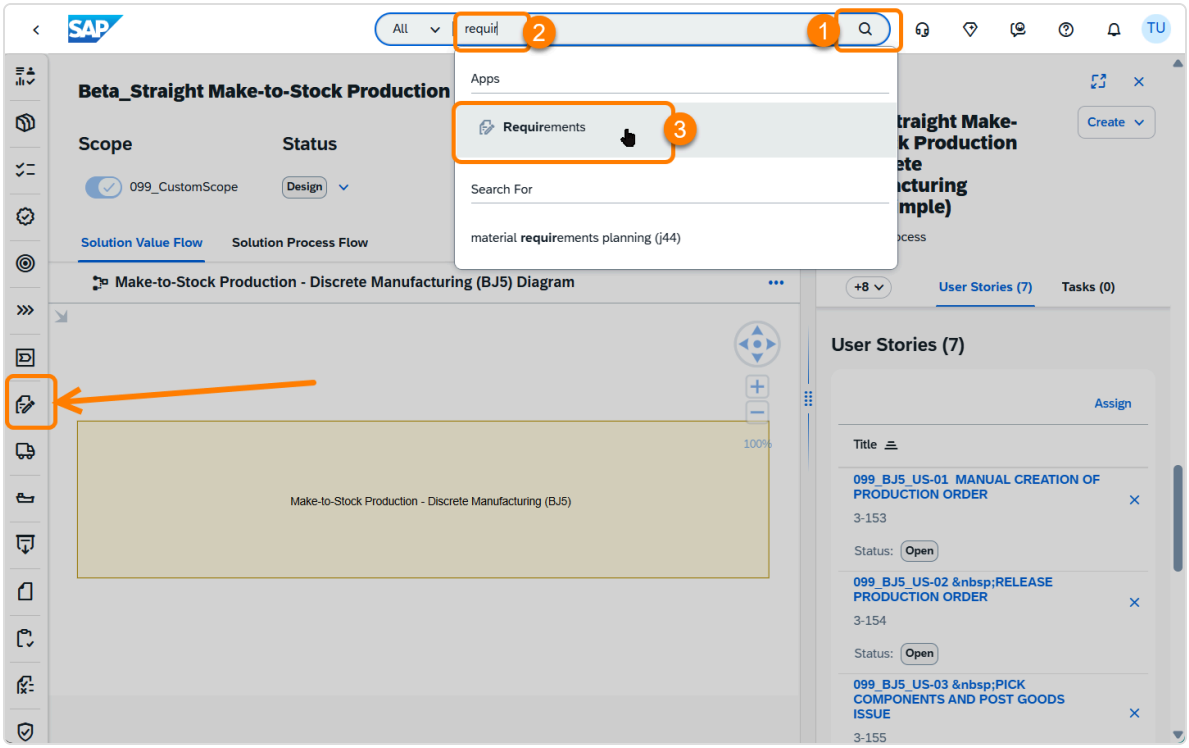
In this step you'll use the *Requirements* app to generate a *Requirements* document from an interview transcript. A BJ5 process expert was interviewed and the recording transcribed; the transcript is waiting for you in the system. The AI will read through it and extract the

key business rules, process steps, and constraints into a structured *Requirements* document, automatically.

---

1. Navigate to the **Requirements** app. There are two ways:

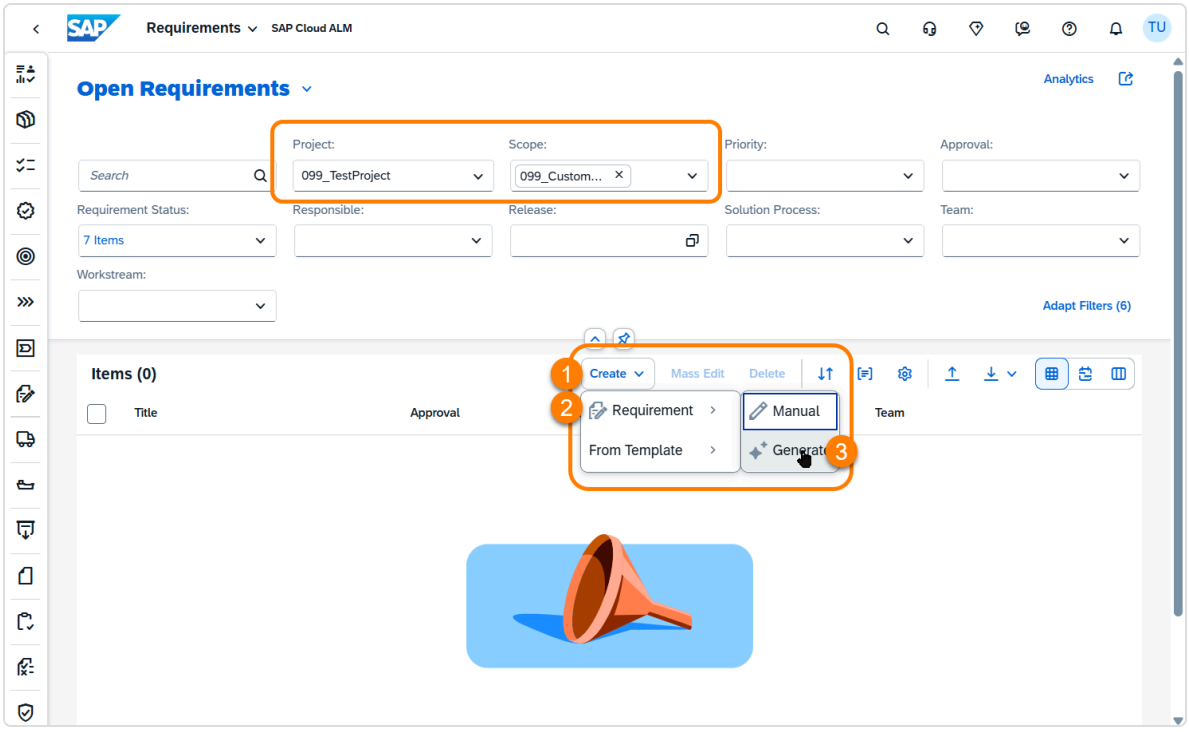
- **Search bar** (top center): type `requir` and select **Requirements** from the quick results
- **Left toolbar**: click the quick-link icon that looks like a document being written on with a pen



18 – Requirements app

The *Requirements* app opens with filters pre-set to your project and scope; confirm both show your workstation number (`###_TestProject` / `###_CustomScope`). The list is empty: no *Requirements* exist yet.

Click **Create**, then hover over **Requirement**: a sub-menu appears with two options. Select **Generate** to have AI build the *Requirement* from a source document rather than writing it manually.

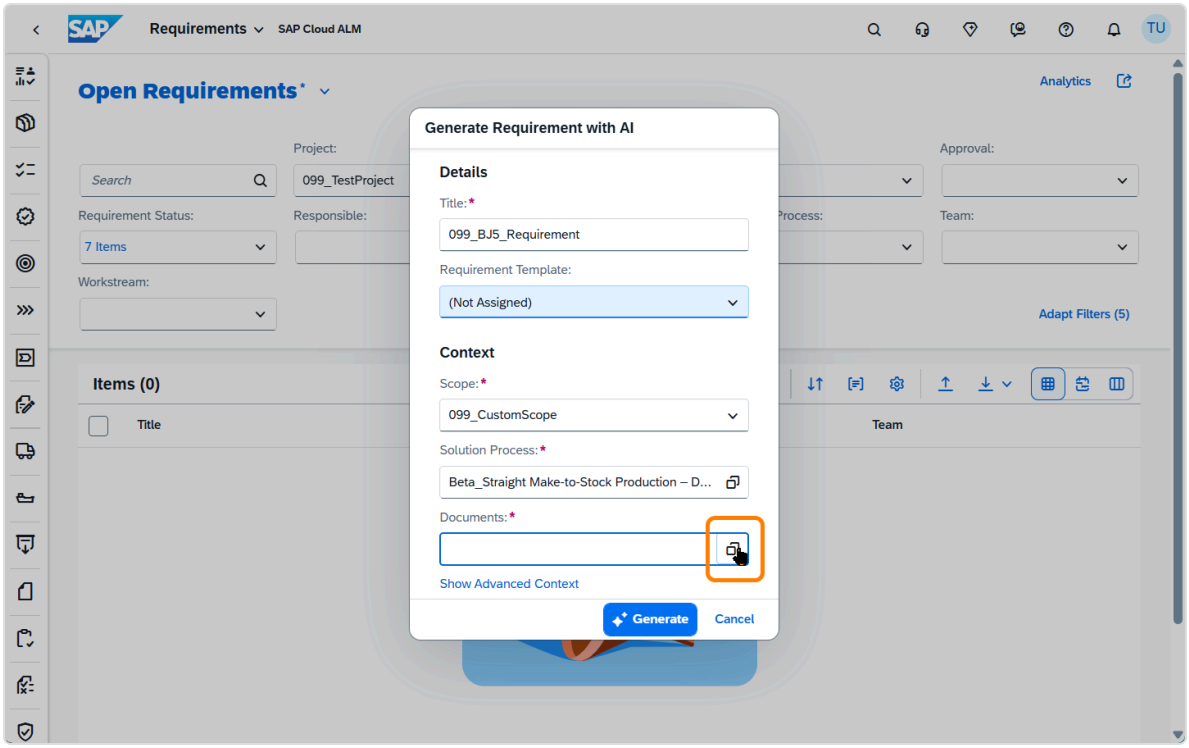


19 – Create → Generate menu

2. The **Generate Requirement with AI** dialog opens. Fill in the fields:

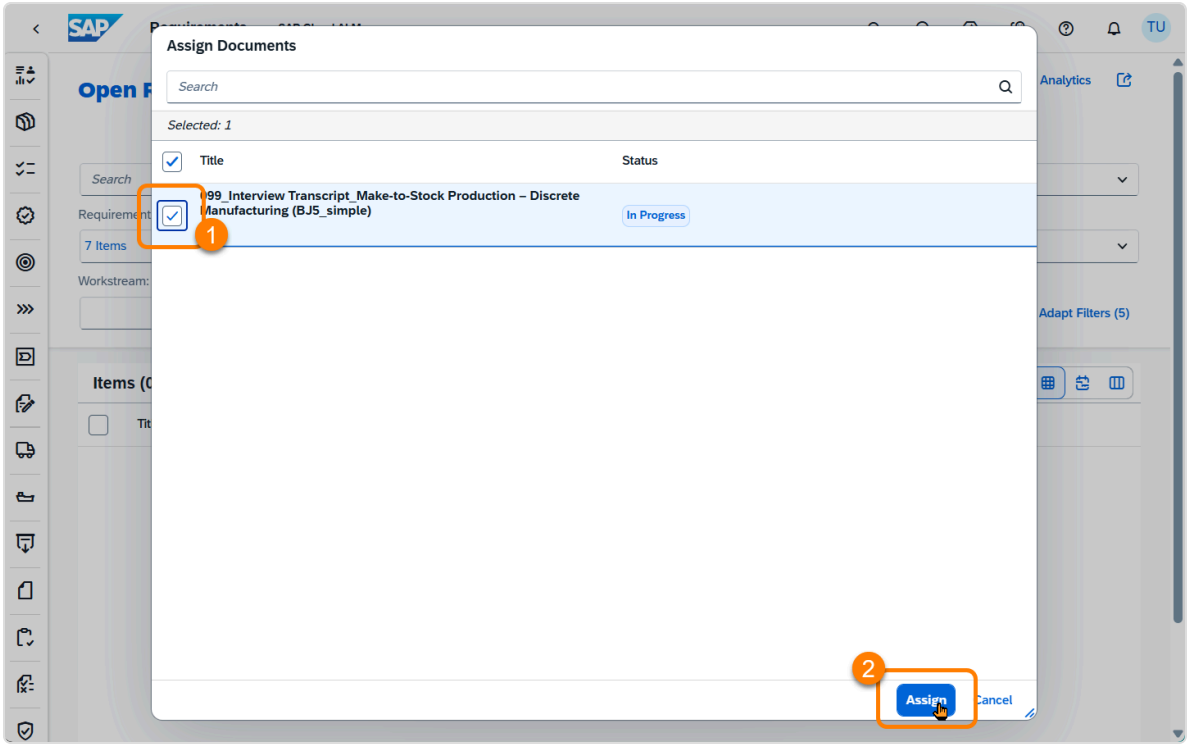
Field	Value
Title	`###_BJ5_Requirement` (replace ### with your workstation number, e.g. `018`, `033`)
Requirement Template	(Not Assigned)
Scope	`###_CustomScope`
Solution Process	Beta_Straight Make-to-Stock Production – Discrete Manufacturing (BJ5_simple)
Documents	← click the selection icon to open the document picker

In the **Documents** field, click the **selection icon** to choose the source document.



20 – Generate Requirement with AI dialog

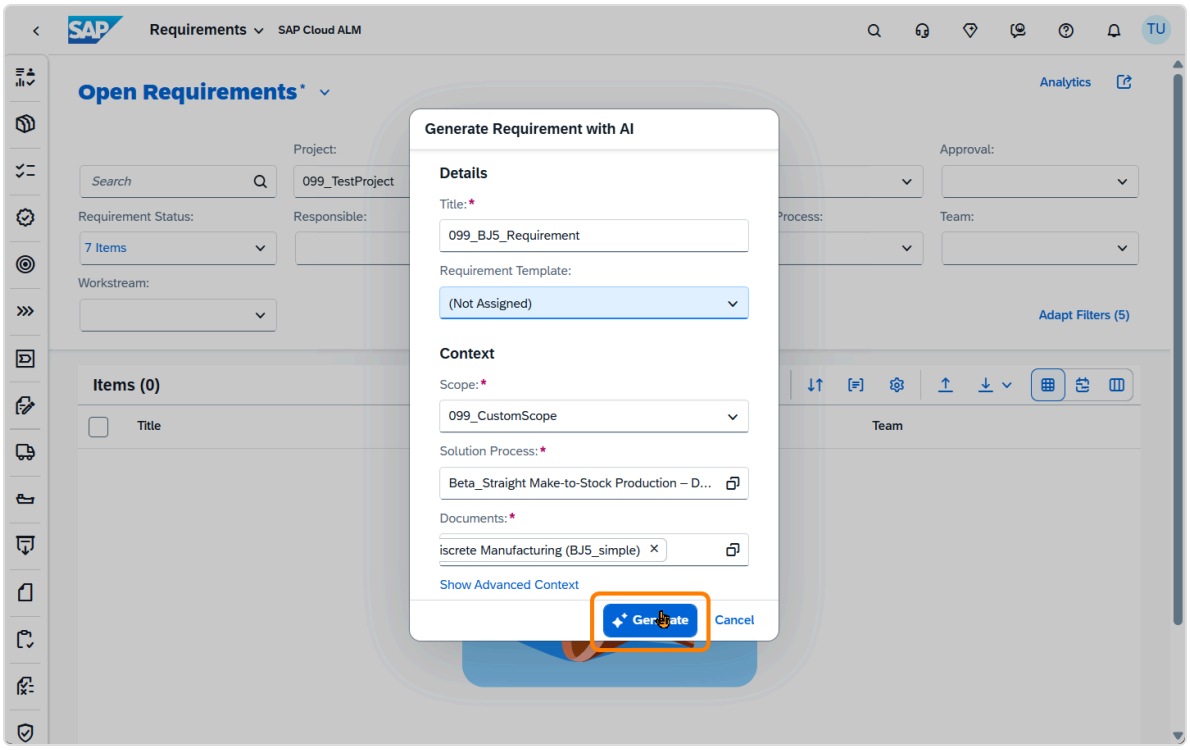
The **Assign Documents** dialog opens. You should see exactly one document: a transcript of a ~20-minute interview with a BJ5 process expert, covering process steps, role responsibilities, and business rules. Select it and click **Assign**.



21 – Selecting the interview transcript

3. Back in the **Generate Requirement with AI** dialog, the transcript now appears in the **Documents** field. Everything is set. Click **Generate**.

SAP Cloud ALM's built-in AI agent reads the full conversation transcript and converts it into a structured *Requirements* document, automatically. You'll be able to review the result in the next screen.



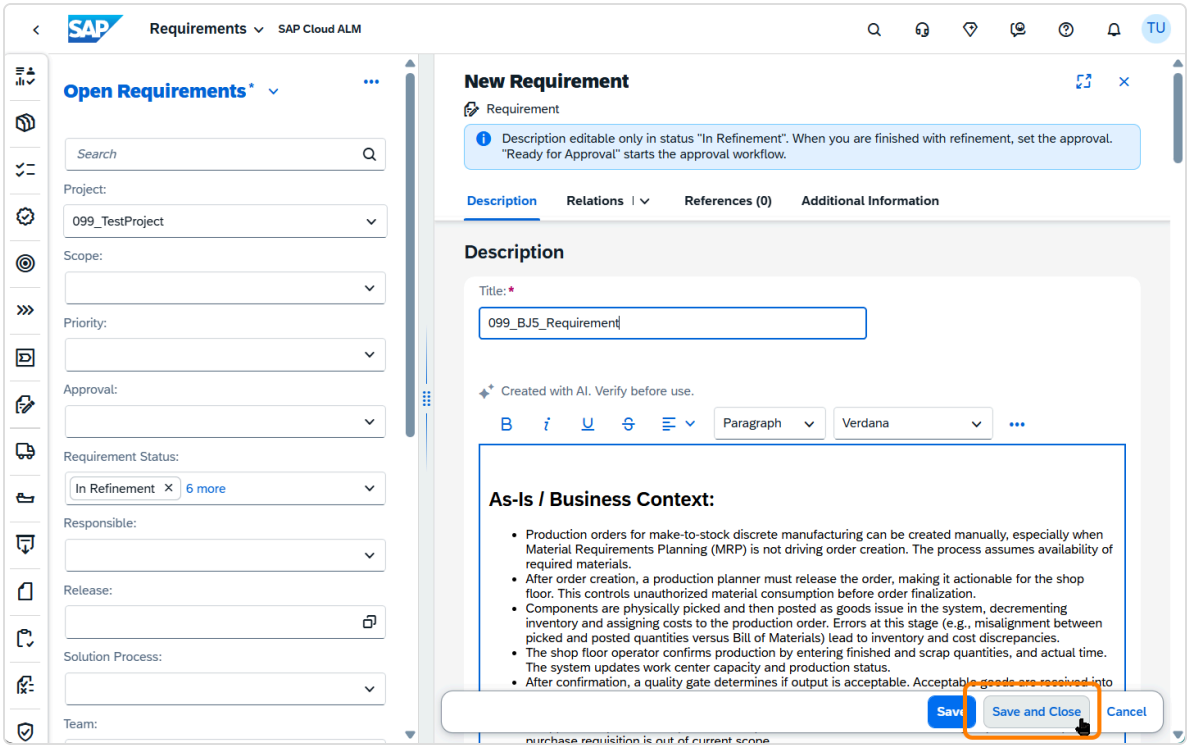
22 – Triggering AI generation

4. The AI has created the new *Requirement*. The detail view opens automatically.

A few things worth noting here:

- Review the generated **description**: this is the structured *Requirements* content extracted from the interview transcript.
- The info banner at the top reminds you that the description is editable while the status is *In Refinement*. Feel free to make adjustments if needed.
- In the left panel, **scroll down** to the **Solution Processes** section: the *BJ5_simple* process has been **automatically linked**, no manual assignment needed.
- At the bottom you can optionally fill in approver, priority, and other fields; skip this for now.

When satisfied, click **Save and Close**.



23 – Generated requirement: detail view

Summary

Chapter 2 complete. The process now has everything the AI agents need:
- [x] BPMN process diagram (from Signavio)
- [x] 7 *User Stories* (assigned in Step 2.2)
- [x] *Requirements* document (generated in Step 2.3)

A 20-minute expert interview was converted into a structured *Requirements* document, automatically linked to the BJ5 process. Time to head back to Joule and try what failed in Step 1.3; this time with proper input in place.

Continue to Chapter 3 – Scope with AI

Chapter 3 – Scope with AI

Prerequisite: You have completed Chapter 2; the *BJ5_simple* process has linked *Requirements* and *User Stories*.

What You'll Do

In this chapter you'll trigger the **Scoping Agent**: an AI agent that reads your linked process documentation and proposes a structured *Test Case* scope. While it works in the background, you'll run a quick manual scoping exercise; when the result arrives, you can see exactly what the AI found versus what you identified.

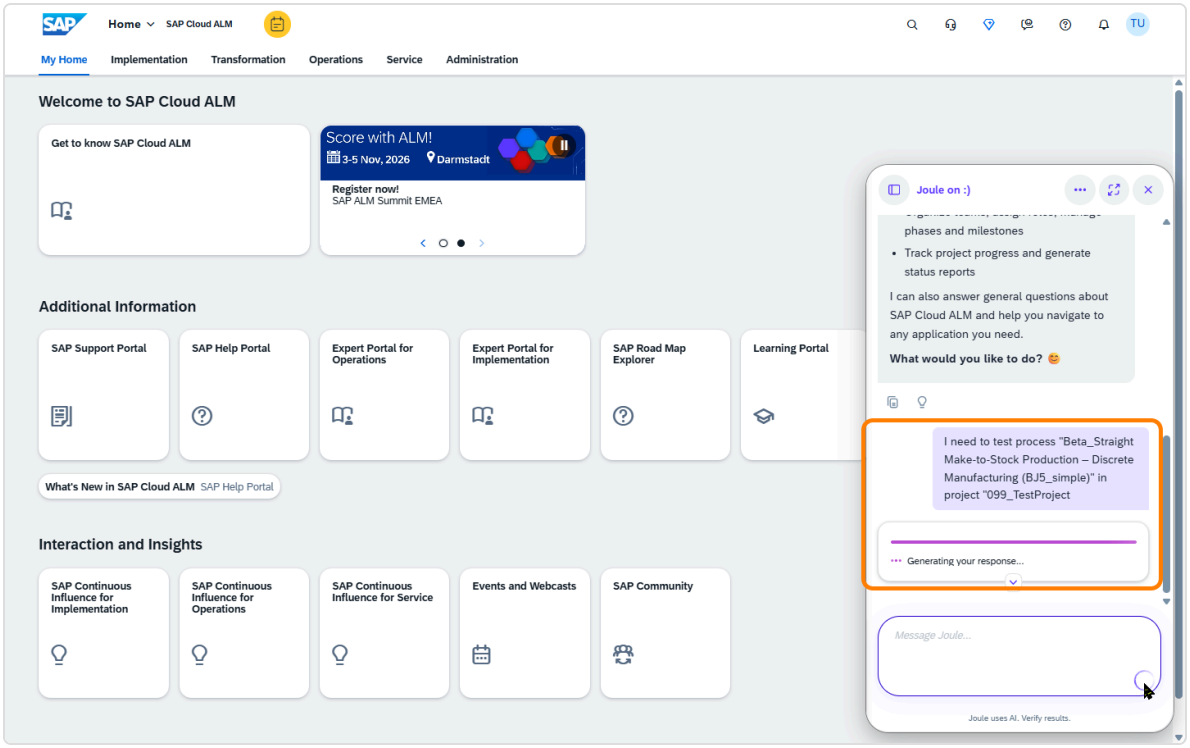
3.1 – Trigger the Scoping Agent

Time to try again; with proper input now in place.

1. Open a new conversation in Joule and send the **same prompt** as in Step 1.3:

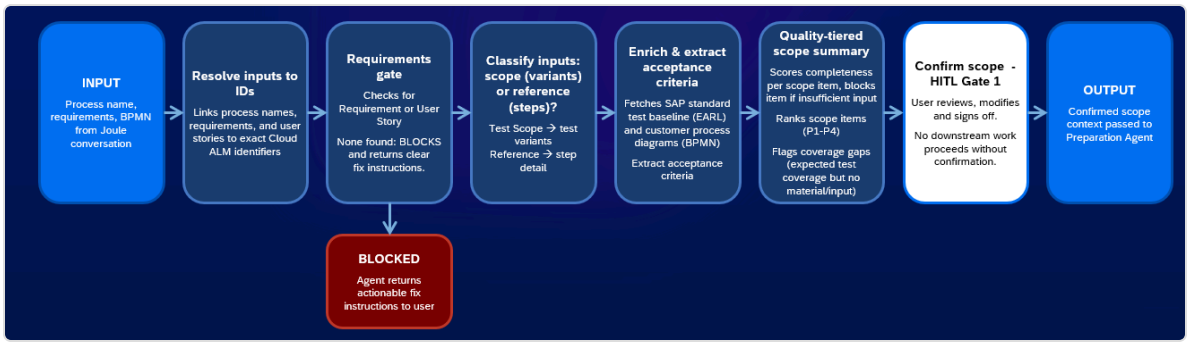
> *"I need to test process "Beta_Straight Make-to-Stock Production – Discrete Manufacturing (BJ5_simple)" in project "###_TestProject""*
>
> Replace **###** with your workstation number.

2. The scoping agent starts working in the background. You'll see a notification or progress indicator.



24 – Scoping agent running

3.2 – What's happening in the background?



25 – Scoping agent methodology

> **The scoping agent applies a structured methodology:**
> 1. It resolves the process name, *Requirements*, and *User Stories* to exact identifiers in SAP Cloud ALM
> 2. It validates that the required input exists; if not, it blocks and returns clear instructions on what to fix
> 3. It classifies each input: process scope variants or detailed reference steps
> 4. It enriches the scope with SAP standard test baselines and extracts Acceptance Criteria from your linked content
> 5. It scores each *Scope Item* for completeness, ranks items by priority, and flags

> coverage gaps
> 6. It presents the result for your review and sign-off; no downstream work starts without your approval (the "Human in the Loop", or HITL, Gate 1)
>
> This is the same analysis a skilled test architect would do, but compressed from hours into minutes.

3.3 – Manual Scoping Exercise

While you wait for the agent, try a structured manual scoping exercise.

Return to the Processes app and for the *BJ5_simple* process, think through these dimensions (a piece of paper to jot notes works fine):

Dimension	Your Answer
What are the main **process paths** (happy path + variants)?	
What **acceptance criteria** can you identify from the *User Stories* and *Requirements*?	
How many distinct **apps/UIs** are involved?	
How many **roles** participate?	
Which steps have **blocking conditions** worth testing?	
What **error scenarios** can you imagine that should be covered?	

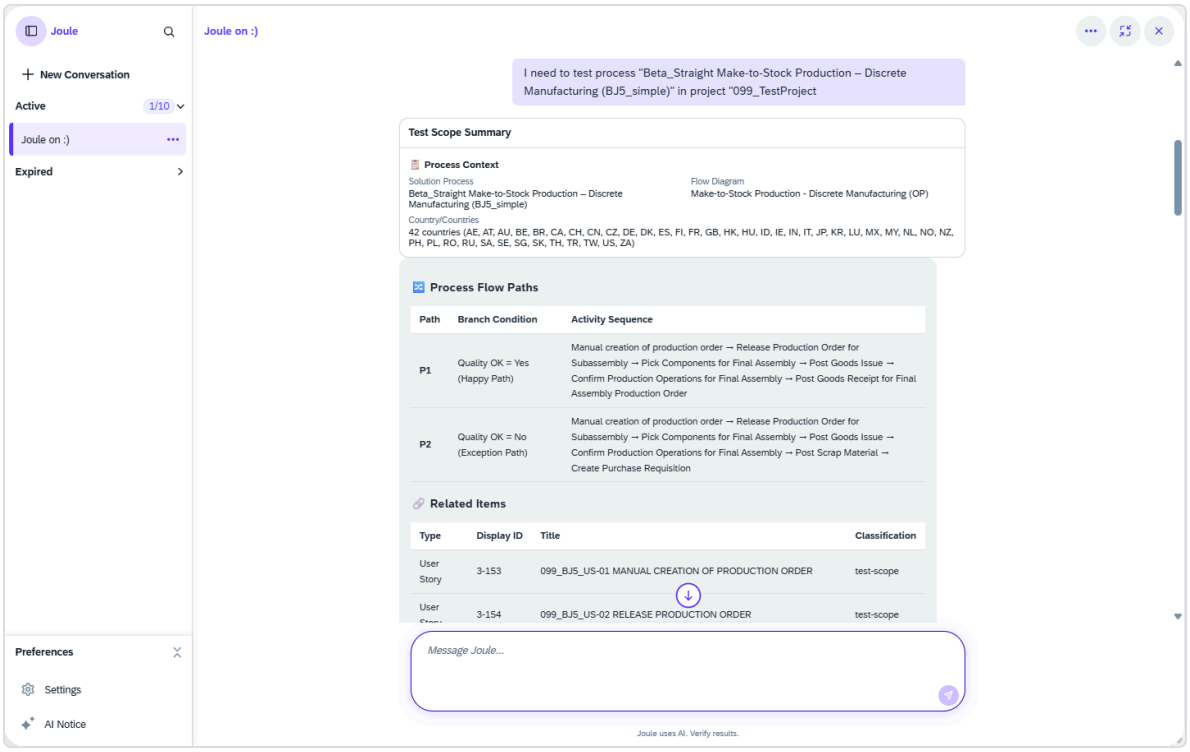
> Hint: The BJ5 process involves 2 roles (Production Planner and Warehouse Clerk) and spans the full make-to-stock cycle from production order creation to finished goods receipt. There is 1 critical quality gate where output is assessed before goods receipt, and a separate failure path for scrap and replenishment.

3.4 – Review the Scoping Agent's Work

1. When the agent is ready, it replies directly in the Joule chat window with a **Test Scope Summary**. The response is detailed; best switch Joule to **full-screen mode** first using the **expand icon** in the top-right of the Joule panel.

The response typically gives you:

- **Process context**: the *Solution Process*, flow diagram, and applicable countries it resolved
- A **summary of identified process flow paths**, reflecting any splits or variants in the process structure
- An overview of the **items it ingested** (your *Requirement* and the 7 *User Stories*)
- Related **standard process procedures** retrieved from the SAP Enterprise Architecture Reference Library (EARL)
- A conclusive list of **acceptance criteria** derived from all of the above, augmented with patterns from SAP's standard process baselines



26 – Test Scope Summary in Joule

After the response, the agent asks you whether to confirm the scope as is or revise it.

> **HITL (Human in the Loop) Gate 1: Test Scope Summary.** This is the first checkpoint in Test Management Assistant's agentic workflow. The agent stops, presents its detailed structured analysis, and waits for your review and sign-off. Nothing proceeds until you confirm.

2. Before making that decision, take a few minutes to work through the proposed scope. For each item, check:
- **Name and description**: does it clearly describe what's being tested?
- **Linked acceptance criteria**: is it traceable back to a *User Story* or *Requirement*?
- **Role separation**: are Production Planner tasks and Warehouse Clerk tasks kept in separate *Test Cases*?

- **Coverage**: does the overall set cover the happy path (P1), the quality gate failure path (P2), and the scrap/replenishment scenario?

> **Compare with your manual notes from Step 3.3:** Did the agent catch everything you identified? Or did it find things you missed? That's exactly the kind of task AI agents excel at: ingesting text, inferring structure, and surfacing patterns at speed.

3.5 – Refine the Scope in Joule

Let's not just accept the scope as is. Joule lets you refine it through conversation, and this is the moment to apply your focus on role switches and shape the scope accordingly.

> **Note for participants sharing a system:** In a real-life scenario you'd typically start with the full E2E process before narrowing down. The reason for this tweak is twofold: first, it lets you experience how to shape agent output through a simple conversation; and second, the agents run on shared infrastructure and we want to avoid overloading the system with heavy concurrent calls. So we recommend limiting the scope to **user stories 5, 6, and 7** (the quality gate, goods receipt, and failure path), where the handoff from Production Planner to Warehouse Clerk happens.

Two options are available; as you'll see, the second is the more direct one.

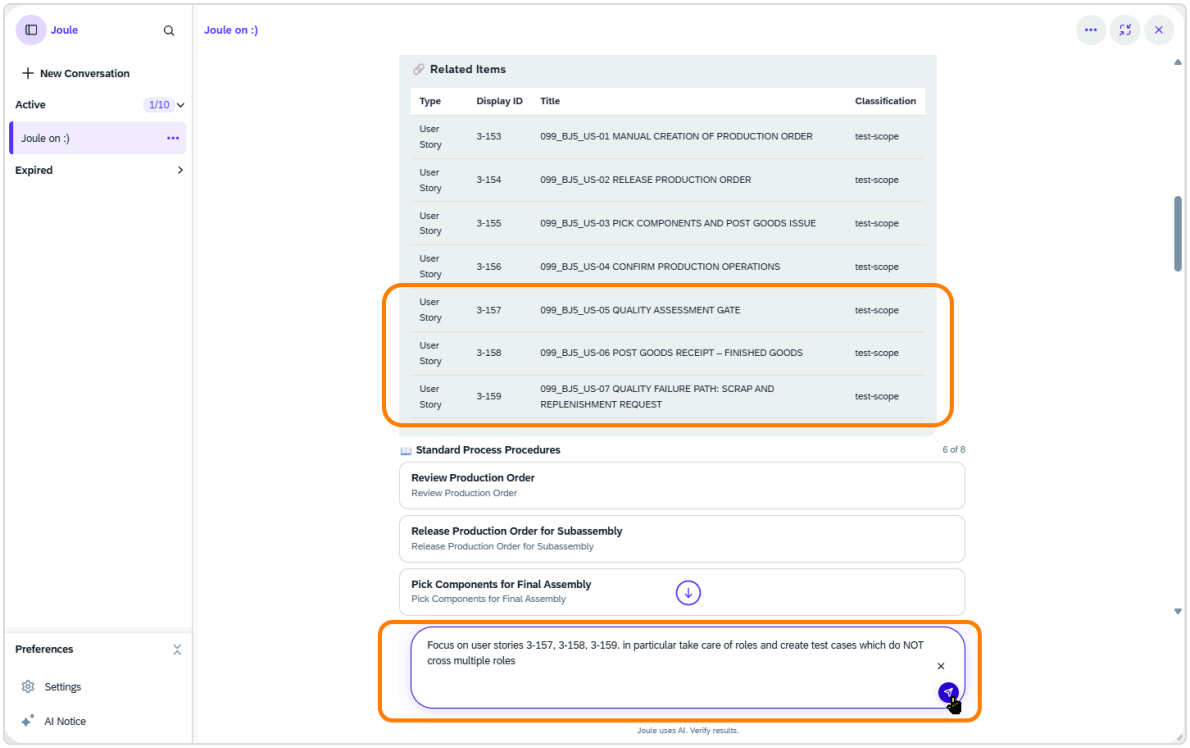
Option 1: Use the built-in "Revise Scope" shortcut

Click the **Select** button in the Joule output panel, then choose **Revise Scope** from the dropdown. This sends a built-in prompt ("No, let me revise the scope" or similar) to the agent, which then asks you what you'd like to change. You'd still need to type your refinement in the follow-up message, making it identical to Option 2 but with one extra round-trip.

Option 2: Write your own prompt

You can also type your refinement directly in the Joule chat window. Navigate to *User Stories* US-05, US-06, and US-07 and note their Display IDs (format: `3-xxx`), then use a prompt like:

> "Focus on *User Stories* **[enter the Display IDs]**. In particular take care of roles and create *Test Cases* which do NOT cross multiple roles."



27 – Refinement prompt: focusing on three user stories

You may try other refinements too:
- **Narrow the scope:** "Remove *User Stories* 1–4 from the scope for now; focus only on the quality gate, goods receipt, and scrap path."
- **Narrow by role:** "Drop the Production Planner for now, and consider only the process steps relevant for the Warehouse Clerk."

The agent re-runs its scoping logic and returns faster than the first pass. It presents the revised scope and pauses for your sign-off again. This time, click **Confirm** to accept the scope and move to the next stage.

Summary

The scoping agent analyzed all your linked process artifacts and proposed a structured *Test Case* scope, with traceability to *Requirements* and *User Stories*, in the time it took you to sketch a rough outline manually.

You now have a **confirmed test case scope** that moves automatically into test planning.

Continue to Chapter 4 – Plan with AI

Chapter 4 – Plan with AI

Prerequisite: You have completed Chapter 3; a confirmed *Test Case* scope exists for the BJ5 process.

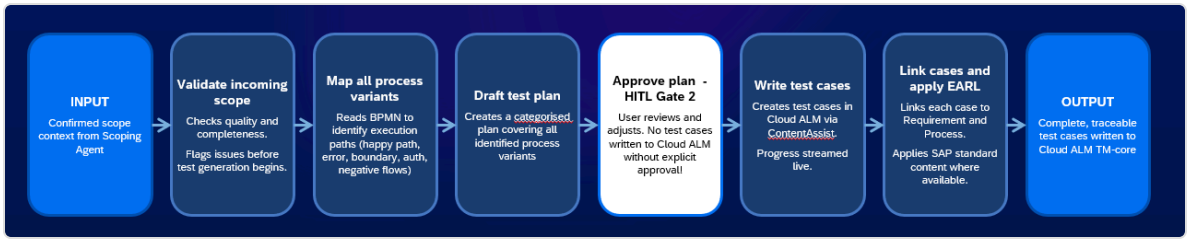
What You'll Do

The scope you confirmed at the end of Chapter 3 has been handed to the **Preparation Agent**, which is already working in the background. This chapter covers the first of its two phases: planning of the *Test Plan* and its *Test Cases*, up to the point where you review and confirm the plan. While the agent works, you will learn what it is doing and why it matters.

4.1 – The Preparation Agent: Planning Phase

The Preparation Agent is a two-phase agent with a human review gate between the phases. This chapter follows Phase 1: from your confirmed scope to a proposed *Test Plan*. Phase 2 (*Test Case* generation) is covered in Chapter 5.

> **The challenge:** Similar to scoping, *Test Case* preparation relies on expertise that cannot scale. *Test Case* volumes grow into the hundreds per project; without structured reuse, duplicates accumulate across teams. Under time pressure, process variants get skipped: happy paths are covered, but error paths, authorisation checks, and boundary cases drop out. Acceptance Criteria end up patchily defined; tests pass but do not prove what they claim to verify.
>
> For each *Test Case* a skilled analyst would need to determine the right application, define preconditions, sequence steps at the correct level of detail, assign a role, and link everything back to *Requirements*, across an entire business process.
>
> **The solution:** The Preparation Agent maps execution paths and Acceptance Criteria from the process model into complete *Test Cases*. It optimises for coverage and *Test Case* efficiency; the expert shifts from producing to reviewing, and covers scope more reliably.



28 – Preparation Agent: pipeline up to HITL Gate 2

> **What the Preparation Agent works through:**
> 1. **Validate incoming scope:** checks quality and completeness of the scope from the Scoping Agent; flags issues before test generation begins
> 2. **Map all process variants:** reads the BPMN diagram to identify every execution path: happy path, error flows, boundary conditions, and negative flows
> 3. **Draft test plan:** creates a categorised plan covering all identified process variants; this is what you will review and approve
> 4. **HITL Gate 2 (Approve plan):** no *Test Cases* are written to Cloud ALM without your explicit approval; you review and adjust before the agent continues into Phase 2

⏱️ While the agent works: try it yourself

Pick one of the *Scope Items* from Chapter 3 (for example: "Confirm Production Operations") and try to write a *Test Plan* entry: a clear description (1–2 sentences), the role performing it (Production Planner or Warehouse Clerk), the preconditions that must be true, and a rough sequence of Test Steps.

How long does that take for one item? Now multiply by the full scope, and note that this is still not the final, well-formed set of *Test Case* steps. That is what *Test Plan* preparation normally looks like.

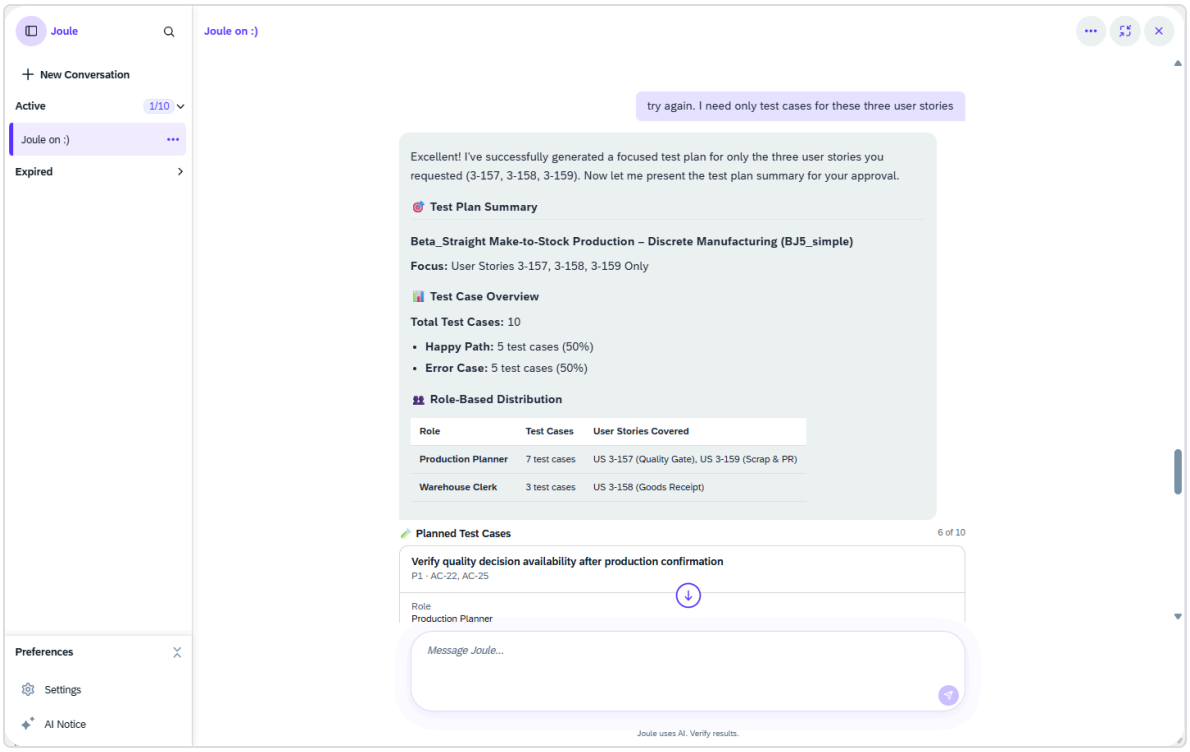
4.2 – Review the Agent's Test Plan

By now the Preparation Agent should have returned with a *Test Plan* proposal. Switch back to your Joule conversation and scroll to find its response.

When Joule responds successfully, it presents a structured **Test Plan Summary**. This may come in slight variations, but typical components and structure are:

- **Test Case Overview**: total number of *Test Cases*, split by path type (happy path vs. error case)
- **Role-Based Distribution**: how *Test Cases* are distributed across roles (Production Planner and Warehouse Clerk) and which *User Stories* each role covers
- **Planned Test Cases**: a deeper look at each proposed *Test Case*, with its process path, linked Acceptance Criteria, and assigned role

- **Acceptance Criteria Coverage**: a summary showing how many Acceptance Criteria from each *User Story* are addressed
- **Key Features**: summary of structural properties the agent applied, such as role isolation, balanced path coverage, and no cross-role tests



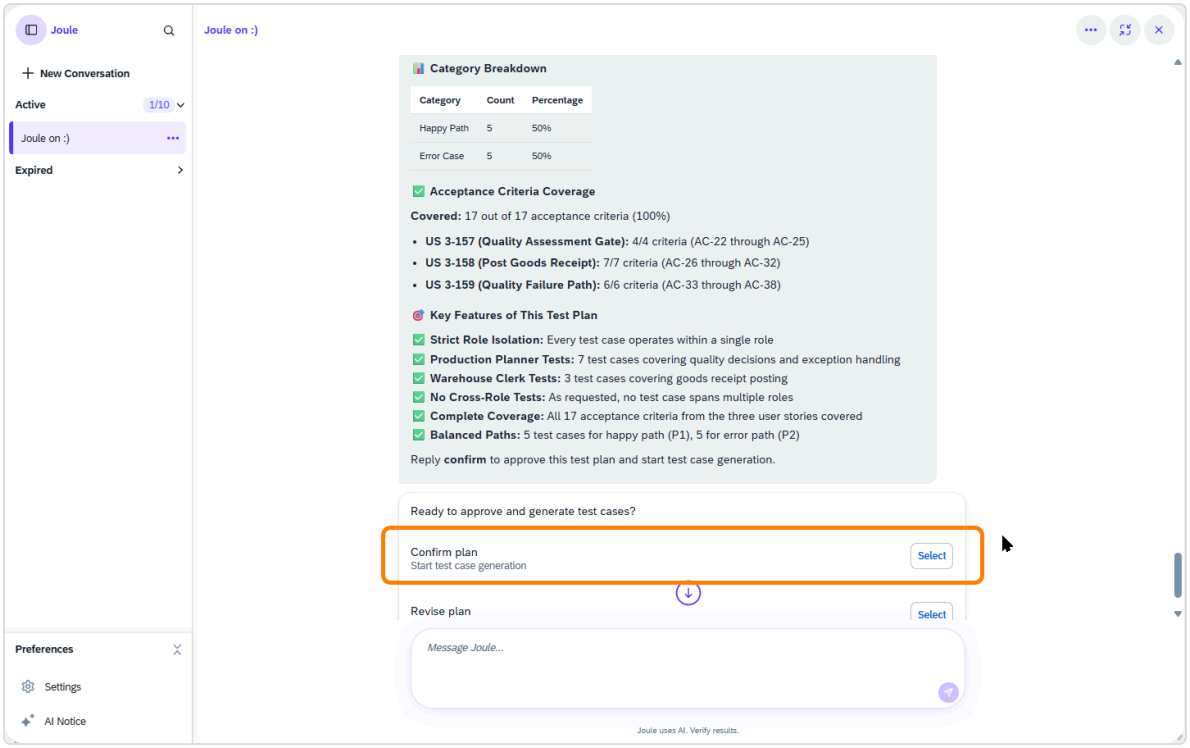
29 – Test Plan Summary: overview, role distribution, and planned test cases

▶️ **Your task:** Review the plan and consider:
- Does the number of *Test Cases* feel right for the scope you confirmed?
- Are roles properly separated? No *Test Case* should span two roles.
- Are both the happy path (P1) and the error path (P2) represented?
- Do the Acceptance Criteria align with what you saw in the *User Stories*?

> **Compare with your try-it-yourself notes from Step 4.1:** Did the agent structure the plan differently than you expected? Did it catch anything you missed?

Review (HITL) Gate 2: Approve the Test Plan

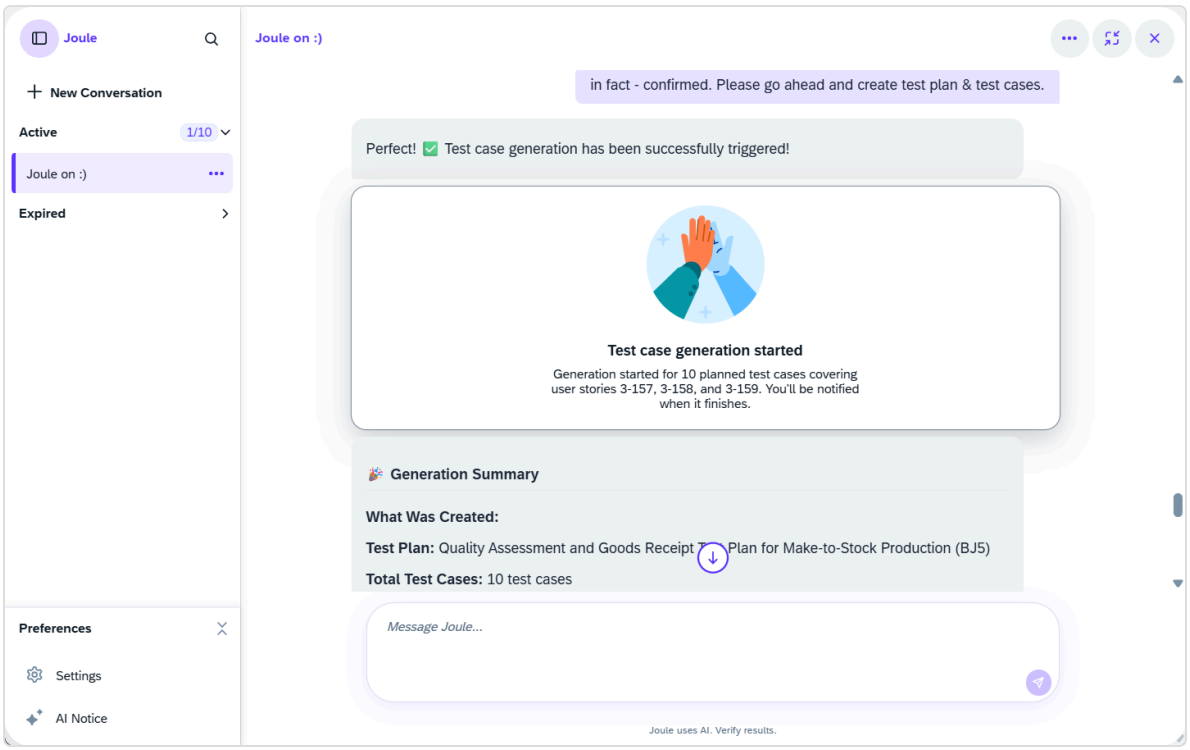
The agent will not start writing *Test Cases* until you confirm the plan.



30 – HITL Gate 2: confirm or revise the test plan

Click **Select** next to **Confirm plan** to approve and start *Test Case* generation.

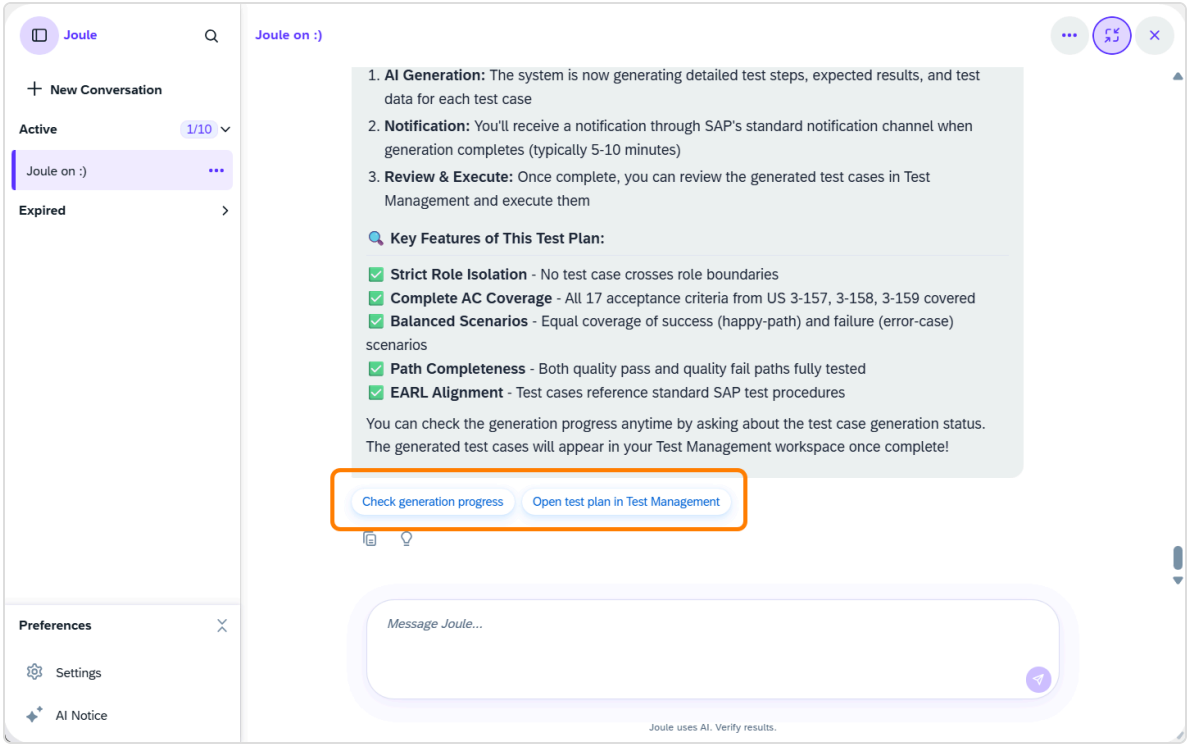
The agent confirms that *Test Case* generation has started, acknowledging your approved plan and the *User Stories* it will work on.



31 – Test case generation started

Joule provides a rundown of what was created and what happens next:
- A **generation summary**: *Test Plan* name and total *Test Cases*
- **Key features** of this *Test Plan*: role isolation, complete Acceptance Criteria coverage, balanced path scenarios, and EARL alignment
- **Next steps**: you will receive a notification when generation is complete (typically 5–10 minutes), after which you can review and execute the *Test Cases* in the native Cloud ALM Test Management app (the next exercise covers this)

Joule also provides quick-access prompts; common options are **Check generation progress** and **Open test plan in Test Management**, which opens the *Test Plan* in the native Cloud ALM app for review. Follow up if you like, or continue to the next exercise.



32 – Generation summary and next steps

Summary

The Preparation Agent has completed its planning phase and started *Test Case* generation:
- [x] *Test Plan* reviewed and approved (HITL Gate 2)
- [x] *Test Case* generation triggered for several *Test Cases* across *User Stories* 3-157, 3-158, 3-159
- [ ] *Test Cases*: generation in progress; you will be notified when ready (see next exercise)

Continue to Chapter 5 – Generate Test Cases

Chapter 5 – Generate Test Cases

Prerequisite: You have completed Chapter 4; a confirmed *Test Plan* exists for the BJ5 process, and *Test Case* generation has been triggered and is running in the background.

What You'll Do

In this chapter you'll follow the **Test Case Generation Agent** as it takes the approved *Test Case* skeletons and generates detailed, executable Test Steps. While it works, you can try drafting one *Test Case* manually, then compare your result with what Joule produces.

5.1 – Phase 2: Test Case Generation

The Preparation Agent is now in its second phase. Having cleared review (HITL) Gate 2 with your approved *Test Plan*, it is working through the following steps:



33 – Preparation Agent Phase 2: test case generation pipeline

- **Write test cases**: For each planned *Test Case*, the agent creates a complete entry in SAP Cloud ALM via ContentAssist, including Test Steps, expected results, preconditions, and role assignments. Progress is streamed live as *Test Cases* are written.
- **Establish traceability**: Each *Test Case* is linked to the corresponding *Requirement* and *Solution Process*, establishing full traceability.
- **Enrich content**: Where available, the agent enriches the content with SAP standard test procedures from the Enterprise Architecture Reference Library (EARL).
- **Output**: Complete, traceable *Test Cases* written to Cloud ALM Test Management, ready for review and execution.

5.2 – Manual Test Case Work

⏱️ While the agent works: try it yourself

Pick one of the *Test Case* skeletons from Chapter 4's approved plan and try to write it manually before seeing what Joule produces. For the *Test Case* "Post Goods Receipt – Finished Goods" (US-06), fill in this template:

Element	Your Draft
Test Case Name	
Role	Warehouse Clerk
User Story	BJ5_US-06: Post Goods Receipt – Finished Goods
Precondition	Production order confirmed; quality assessment passed (Quality OK = Yes, P1 path)
App	Goods Movements / Post Goods Receipt
Step 1	Open the Goods Receipt app and select the confirmed production order
Step 1: Expected Result	
Step 2	Verify the quantity matches the confirmed production output
Step 2: Expected Result	
Step 3	Select movement type and post the goods receipt
Step 3: Expected Result	
Step 4	Verify the stock update in the storage location
Step 4: Expected Result	
Parameters needed	Production order number, plant, storage location, finished goods material, quantity

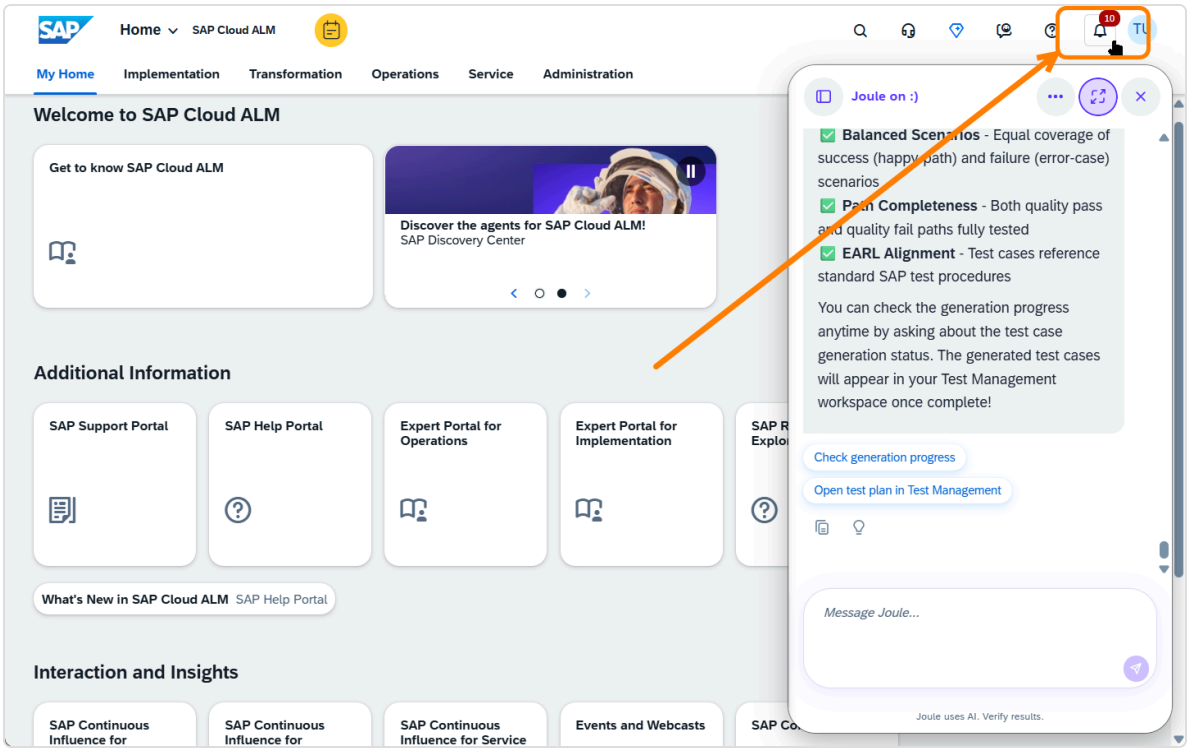
Keep your notes; you'll compare them with what Joule produced in Section 5.3.

5.3 – Review the Generated Test Cases

By now the *Test Cases* should be ready. Time to see what Joule produced.

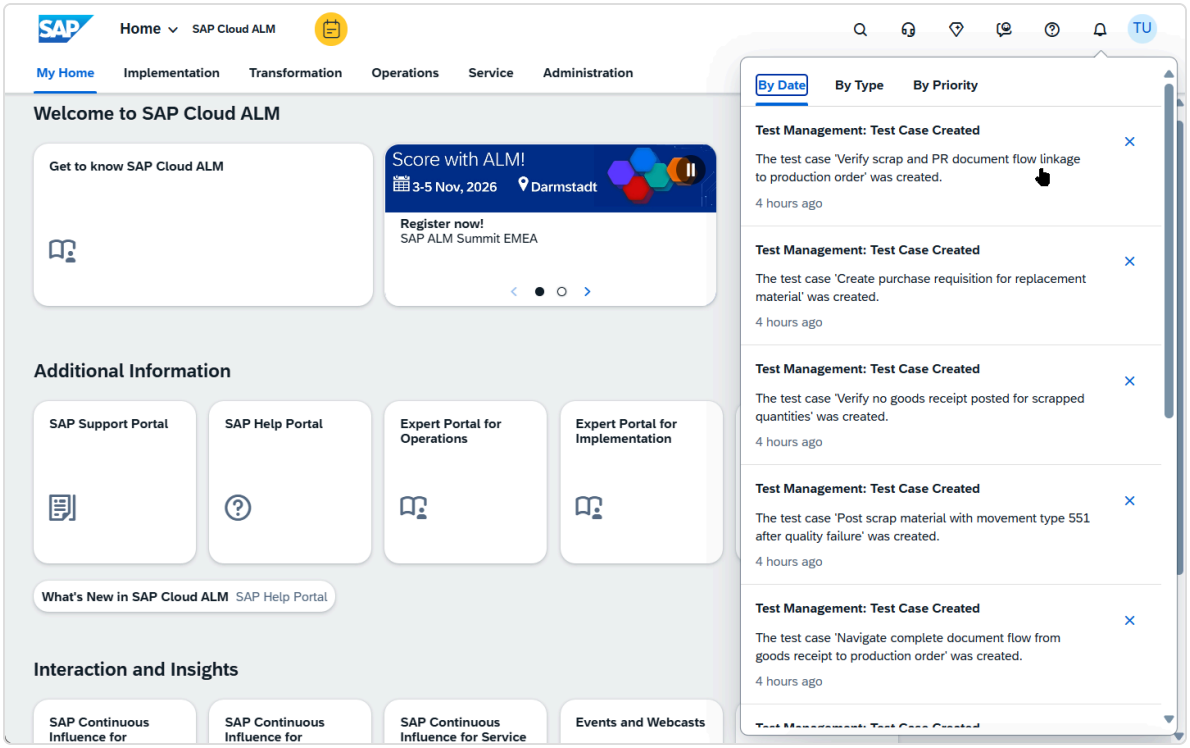
Finding the notifications

While you were working on your manual *Test Case*, Joule was completing *Test Case* generation in the background. SAP Cloud ALM sends an in-app notification for each *Test Case* created. You will see a red bubble badge on the **notifications bell** in the top-right corner of the screen.



34 – Notification badge: test cases created

Close the Joule panel first, then click the **notifications bell**. An overlay opens showing one entry per *Test Case*, with the *Test Case* name and an indication of when it was created.

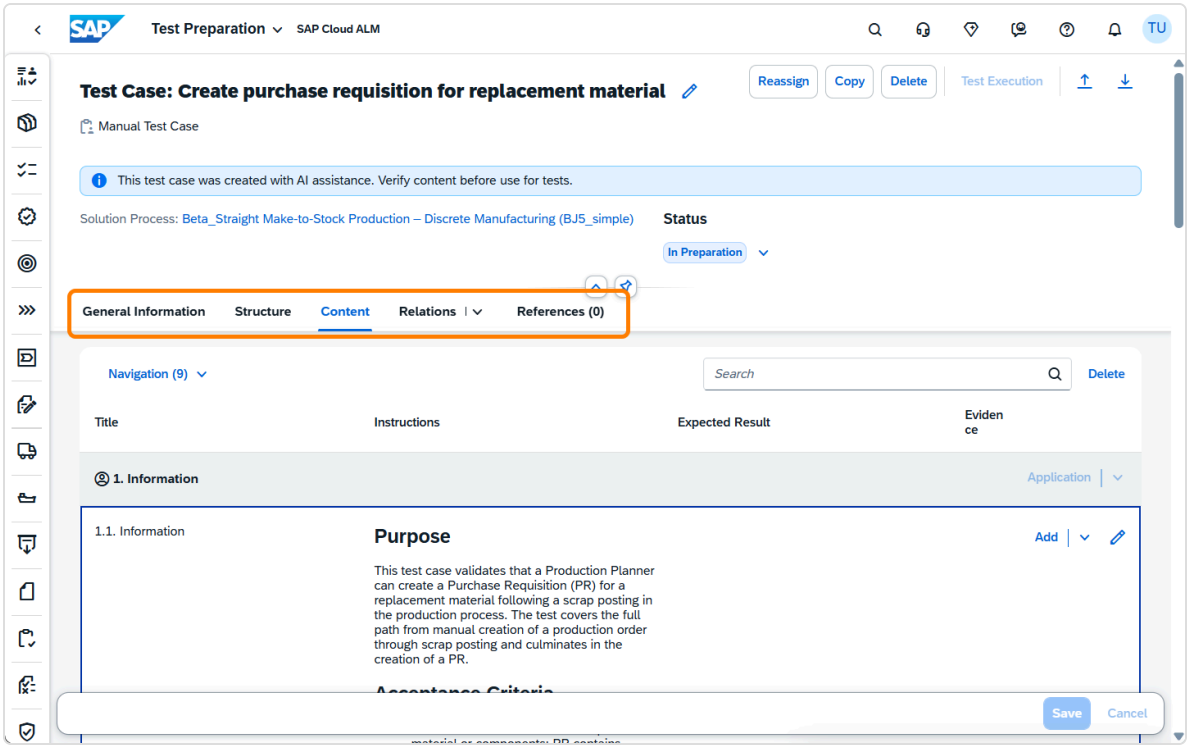


35 – Notification overlay: one entry per generated test case

Opening and browsing a test case

Click on any notification to open the *Test Case* in the SAP Cloud ALM Test Preparation interface. We suggest starting with the first (oldest) *Test Case* in the batch, which is usually the most encompassing end-to-end happy path flow. Click it and the *Test Case* opens in the native Cloud ALM *Test Case* app. Take a few minutes to browse, focusing on:

- **Header area**: indication of a Manual *Test Case*, linking to related artefacts, status "in preparation"
- **General information**: you can find the *Test Case* ID here; note it down, you will need it in a later step
- The **Content** tab: a **Purpose** section describing the business objective, the **Acceptance Criteria** linked from the *User Story*, and the full set of **test steps** with instructions and expected results
- The **General Information** tab: traceability links to the *Solution Process* and *Requirement*, and the source indicator: "Manual *Test Case* – created with AI assistance"
- The **Relations** tab: connection to the *Scope Item* and *Test Plan* entry
- The **References** tab: any EARL standard content applied



36 – Test case detail: Content tab with purpose, acceptance criteria, and test steps

Quality check

▶️ **Your task:** Review the *Test Case* against these criteria:

Quality Criterion	Check
Traceable to a *User Story* or *Requirement*	☐
Correct role assigned	☐
Logical grouping of steps	☐
Clear, actionable steps (not too vague, not too granular)	☐
Expected result for every step	☐
Input parameters identified	☐

> **Compare with your manual draft from Step 5.2:** Did Joule's version match what you wrote? What did it do better? What would you change?

What you have learned

Joule produced several complete, structured *Test Cases*, each with a clear business purpose, traceability to Acceptance Criteria, and a full set of executable Test Steps with expected results; all in the time it took you to draft one manually.

Look closely at the Test Steps: they contain **parameter placeholders** for values the tester will need to enter. Fields like business partner, plant number, production order number, or material number have been identified but not yet filled with actual system values. In some cases, example values from linked artefacts or SAP standard content may appear as a starting point; these are not yet ready for Test Execution. Resolving these parameters is the job of the next agent in the pipeline: the **Test Data Management Agent**, covered in Chapter 6.

Summary

The Preparation Agent's Phase 2 wrote detailed, executable *Test Cases* for the full BJ5 scope; complete with steps, expected results, and parameter placeholders.

- [x] *Test Cases* generated and reviewed
- [x] Quality criteria checked: role assignment, step clarity, traceability, expected results
- [ ] Test data values: parameter placeholders identified; to be verified and resolved in Chapter 6

Continue to Chapter 6 – Generate Test Data

Chapter 6 – Generate Test Data

Good test data may be one of the most underestimated parts of Test Preparation. A single wrong value (an invalid material, the wrong plant, a date outside the planning horizon) is enough to make a test fail for the wrong reasons, wasting time and masking real *Defects*. It is also a recurring burden: even for established *Test Cases*, data values get consumed during Test Execution (orders confirmed, stock moved, documents created) and must be refreshed before the next run. Yet gathering valid, consistent test data for a business process like BJ5 traditionally demands expert knowledge of both the process and the system configuration.

Prerequisite: You have completed Chapter 5; *Test Cases* have been generated for the BJ5 process and are visible in Test Preparation.

What You'll Do

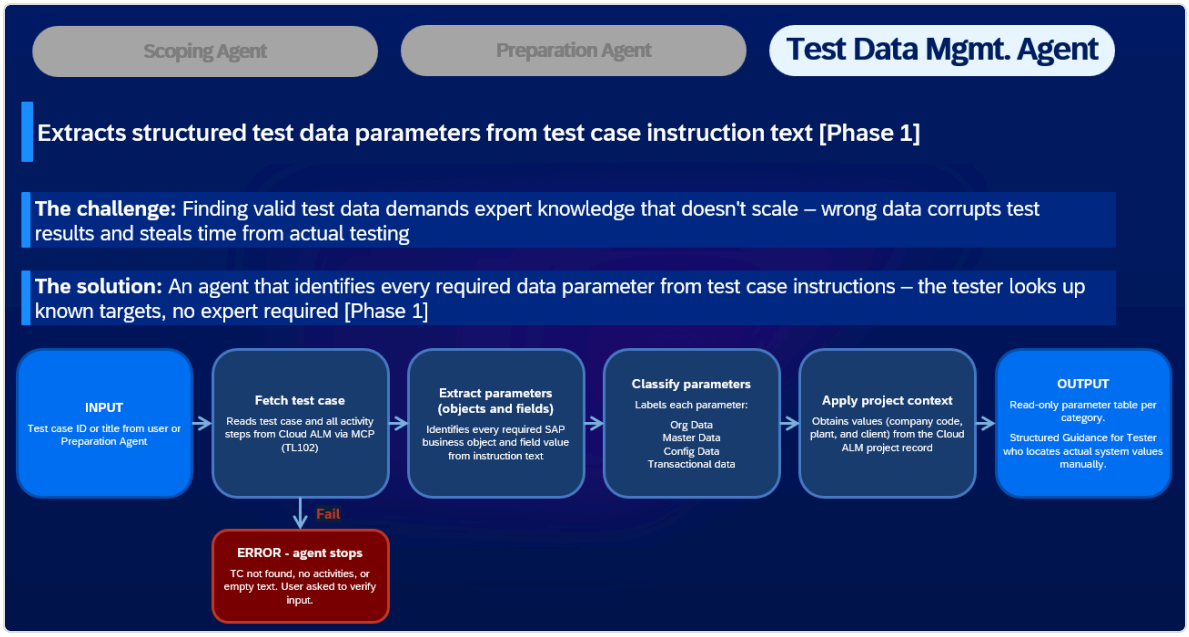
In this chapter you'll work with the **Test Data Management Agent**. In its current Phase 1 form, the agent reads a *Test Case* and extracts every parameter placeholder: the fields where a tester will need to provide a specific value to execute the step. It classifies them by data category; for parameters that are constant across the project (such as company code, plant, and client), it applies the known project-level values automatically so the tester does not need to look them up.

The current Phase 1 capability is the foundation for a progressively richer roadmap. Future phases will connect the agent to a live S/4HANA system: first to suggest values that actually exist in the test environment, then to propose new master data records where the right values do not yet exist, modelled on patterns from a productive system. The final step will extend to writing data directly into target systems; not only master data but also transient data: the prerequisite document chains (inquiry, offer, order, delivery) needed for *Test Cases* that begin mid-flow in a business process.

For now, Phase 1 turns unstructured Test Step text into a structured, actionable parameter checklist.

6.1 – Understanding the Agent

The diagram below shows the full pipeline of the Test Data Management Agent in Phase 1:



37 – Test Data Management Agent: Phase 1 pipeline

What the Test Data Management Agent works through:

1. **Fetch the Test Case:** reads the *Test Case* and all its activity steps from SAP Cloud ALM; if the *Test Case* is not found or has no activities, the agent stops and asks you to verify the input
2. **Extract parameters:** reads every Test Step and identifies each place where a specific value is needed; each parameter is named, typed, and described
3. **Classify:** groups each parameter into one of four categories: Organisational data, Master data, Configuration data, or Transactional data
4. **Apply project context:** fills in known fixed values from the project configuration (company code, plant, client), reducing manual lookup for values shared across all *Test Cases*
5. **Output:** a structured parameter table per category, with descriptions, example values where available, and dependency reasoning

Phase 1 scope note: The agent identifies and classifies parameters; it does not yet look up or suggest actual system values. The tester uses the output as a structured checklist.

6.2 – Identify the Parameters in Your Test Case

⏱️ While the agent works: try it yourself

Open one of your generated *Test Cases* in Test Preparation, go to the **Content** tab, and read through the Test Steps. For each step, note every parameter: every place where the tester will need to enter a specific value.

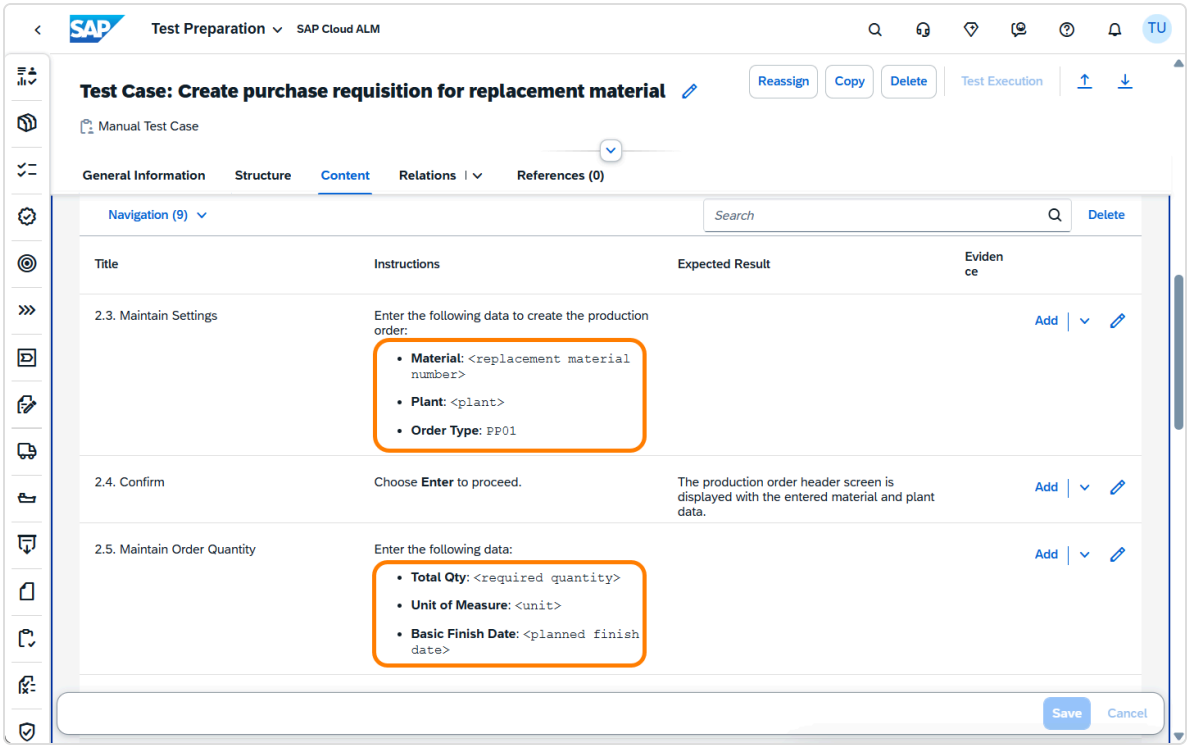
Look for patterns like these:

- **Material**: `<replacement material number>`; must exist in the system
- **Plant**: `<plant>`; the plant where the production operation takes place
- **Order Type**: PP01; a configuration value, already fixed here
- **Total Qty**: `<required quantity>`; must be valid for the material and order, and the quantity must be available in a storage location so the picking steps can be executed
- **Basic Finish Date**: `<planned finish date>`; must fall within the planning horizon

For a make-to-stock production and warehousing process like BJ5, you'll typically also encounter:

- **Sales Area**: a combination of Sales Org, Distribution Channel, and Division, tied to a specific company code; all fixed from the project configuration
- **Work center**: must exist in the plant and be assigned to the correct cost center
- **Storage location**: must be valid for the plant and movement type
- **Production order number**: a transactional reference that only exists after the order is created
- **Confirmed quantity**: depends on the output of a prior step

Note the distinction: some values are fixed (company code, plant), some are looked up once before the test (material, work center), and some are only generated at runtime (production order number). This is exactly the classification the agent applies.



*38 – *Test Case* parameters: placeholder values in the instruction text*

6.3 – Trigger the Data Agent

Unlike the Preparation Agent covered in previous chapters, the Test Data Management Agent does not start automatically after *Test Case* generation. This is intentional: test data is a separate output, used alongside Test Steps rather than embedded in them. A *Test Case* describes *what* to do; test data provides *with what* to do it. These two artefacts have different lifecycles in SAP Cloud ALM; data will typically be resolved when the *Test Case* is in an advanced status or already assigned to a tester. The decision of when to enrich a *Test Case* with data is often made by the person preparing for execution: initially a key user, and later the tester directly.

There is also a practical reason to start fresh: generated *Test Cases* need to be processed and indexed by Cloud ALM before the agent can read them; starting a new Joule conversation ensures a clean context.

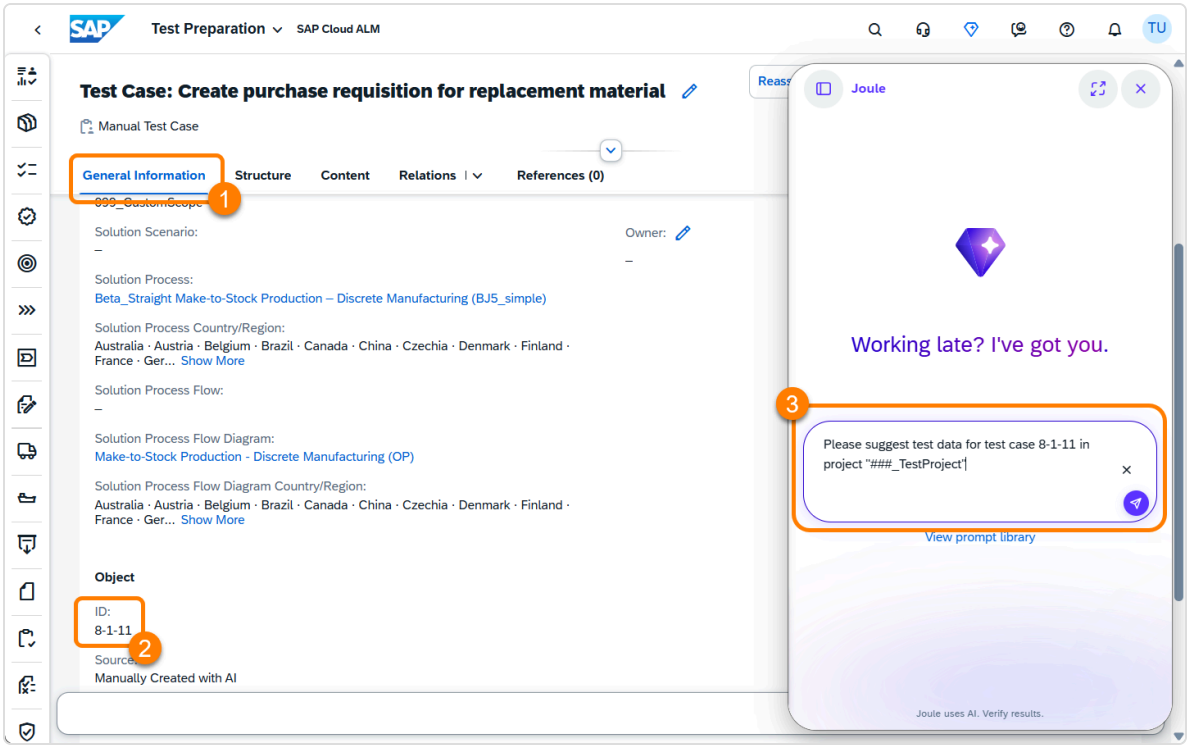
If you noted down the *Test Case* ID in Chapter 5, you are ready to proceed. If not, **retrieve the test case ID** first:
- Switch to the **General Information** tab of the *Test Case* you reviewed in Step 6.2.
- Scroll to the **Object** section at the bottom. Note down the **ID** shown there; it follows the format `8-1-123`.

Triggering the agent

1. Open a **new Joule conversation** (click the new conversation icon or close and reopen the Joule panel).

2. Send this prompt, replacing `###` with your workstation number and inserting the *Test Case* ID noted above:

> *"Please suggest test data for test case [your test case ID] in project "###_TestProject""*



39 – New Joule conversation: triggering the Test Data Management Agent

The agent starts working: it fetches the *Test Case*, reads all activity steps, extracts and classifies every parameter, and applies the project context.

6.4 – Review the Agent's Output

When the agent returns, it presents a structured table of all data parameters identified in the *Test Case*, grouped into four categories.

Organisational data
The fixed organisational frame: sales organisation, distribution channel, division, and dependent org units such as plant and storage location. These values are typically re-usable across *Test Cases* in a project, and the agent applies them automatically from the project context wherever possible.

Master data
Objects that must exist in the system before the test can run: materials valid for the sales area and plant, business partners in the correct role (for example: sold-to party, ship-to, payer), and any dependencies between them. A sold-to party determines which ship-to and payer are valid; a material type determines which item category is auto-assigned.

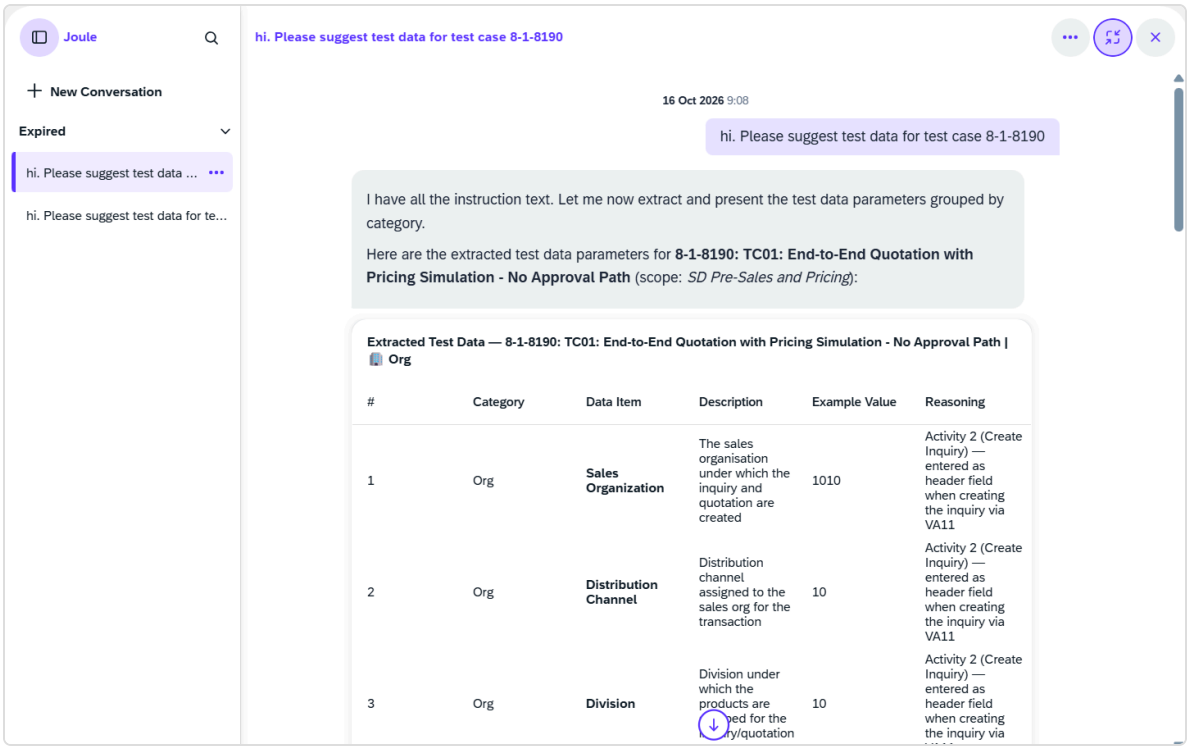
Configuration data
System settings that affect process behaviour: pricing procedures, document types, workflow configurations, authorisation roles. These rarely change between test runs but must be verified to exist and be correctly set up before testing.

Transactional data

Values that are either chosen by the tester at runtime or generated on the fly during execution: quantities, dates, and document numbers (order numbers, delivery numbers, production order IDs) that only exist after a preceding step has been completed.

Why dependencies matter

The agent does not just list parameters; it explains which values drive which others. This is more valuable than it first appears. Consider a quantity of "10" entered in a production order step: taken in isolation it looks arbitrary. Trace it forward through the process and that same quantity must arrive at a picking step where the storage location needs to hold sufficient stock. If it does not, the test will fail for the wrong reasons: not a *Defect*, but a data setup problem. Understanding these dependencies upfront is the prerequisite for future agent phases to run automated checks and resolve such blocking conditions before a test is even started.



40 – Test Data Management Agent output: parameters grouped by category with example values and reasoning

▶️ **Your task:** Compare the agent's parameter table against your manual notes from Step 6.2:

Quality check	What to look for
Completeness	Are all parameter placeholders from the Test Steps in the table? Are there steps where a value is clearly needed but not captured?
Classification	Are categories assigned correctly? Company code = Org Data; material = Master Data; production order number = Transactional Data

Quality check	What to look for
Project context	Are fixed values like plant and company code already populated from the project record?
Reasoning	Does the agent trace each parameter back to the Test Step it came from and explain why it is needed?
Dependencies	Does the output indicate which parameters drive or constrain others? Use the follow-up prompt below if not.

> **Compare with your manual notes from Step 6.2:** Did the agent capture everything you identified? Did it find parameters you missed?

At the end of its response, the agent offers suggestions for next steps; one of the most useful is an analysis of the dependencies between the parameter groups. Try sending one of the offered follow-up prompts, or write your own:

> "*Outline the dependencies between the parameters you identified.*"

The agent should map which values must be consistent with which others; useful both for manual data lookup and as a foundation for future automated data preparation.

Summary

The Test Data Management Agent turned unstructured Test Step text into a structured parameter checklist; classifying every placeholder by data category and applying fixed project values automatically.

- [x] *Test Case* parameters extracted and classified by the agent
- [x] Parameter dependencies and categories understood
- [ ] Actual system values: to be looked up by the tester using the agent's output as a checklist

You have now completed the full AI-assisted Test Preparation cycle:
- [x] *Requirements* generated from interview transcript (Chapter 2)
- [x] Test scope proposed by Scoping Agent (Chapter 3)
- [x] *Test Plan* structured by Preparation Agent: Phase 1 (Chapter 4)
- [x] *Test Cases* written by Preparation Agent: Phase 2 (Chapter 5)
- [x] *Test Case* parameters identified and classified (Chapter 6)

Time saved

To put the session in perspective: the table below shows rough estimates of the manual effort each stage would typically take a skilled practitioner. The AI agents deliver the first draft in minutes; human review adds time, but the net result is already a substantial reduction, and it will grow as the agents become more capable.

Stage	Manual effort (estimate)	AI-assisted
Requirements from transcript	2–4 hours (BA)	~2 minutes + review
Test Case scoping	4–8 hours (architect)	~3 minutes + review
Test Plan creation	2–4 hours (test analyst)	~3 minutes + review
Test Case writing	1–2 days (test analyst)	~5 minutes + review
Test data parameters	2–4 hours (analyst)	~2 minutes + review
Total	**3–5 days**	**< 30 minutes**

Continue to Chapter 7 – Recap & Outlook

Chapter 7 – Recap & Outlook

7.1 – What You Accomplished

You just worked through a complete AI-assisted Test Preparation cycle for a real SAP S/4HANA business process. Here is what you did, and what the agents did alongside you.

Stage	What you did	What AI contributed
Orientation	Explored the BJ5 process in Cloud ALM; assigned *User Stories*; reviewed the process flow	Joule demonstrated the quality gate: no input, no output
Build input	Reviewed the interview transcript; connected *Requirement* and *User Stories* to the process	In-built AI feature generated a structured *Requirement* from a raw transcript
Scoping	Mapped the process manually on paper; evaluated role boundaries and process paths	Scoping Agent analyzed all linked artifacts and produced a traceable test scope in minutes
Test planning	Drafted a *Test Plan* entry manually for one *Scope Item*	Preparation Agent structured the full scope into a *Test Plan* with roles, paths, and Acceptance Criteria
Test cases	Wrote one *Test Case* manually in a template	Preparation Agent wrote complete, executable *Test Cases* for the confirmed scope
Test data	Identified parameter placeholders in a *Test Case* step by step	Test Data Management Agent extracted and classified all parameters, explained dependencies

What would have taken a test team several days, end to end, was completed by the agents in the time it took you to work through the manual exercises. Not as a replacement for your judgment, but as the first draft you review rather than the blank page you fill.

The shift: you moved from *producing* test artifacts to *reviewing* them. The expert role does not disappear; it changes:
Instead of writing Test Steps, you validate whether the agent got the scenario right.
Instead of looking up master data, you confirm whether the classifications make sense.
That judgment is irreplaceable; the mechanical production of structured content is not.

7.2 – What's Coming Next

The capabilities you worked with today are Phase 1 of the Test Management Assistant in SAP Cloud ALM. The roadmap has three main directions.

Scoping Agent: broader input sources

Today the Scoping Agent reads from SAP Cloud ALM. In future waves it will be able to consume additional input types: Jira *Requirements*, Signavio process models, output from other SAP AI assistants in the multi-model architecture (MMA), and more. Scoping will become source-agnostic and will work across the landscape your project actually uses, not just what lives inside Cloud ALM.

Test Data Management Agent: from identification to execution

Today the agent identifies and classifies parameters. The next phases extend this into active data management: reading existing data from connected S/4HANA systems, suggesting values from productive data that fit the test context, and eventually writing clean test data directly into test systems before execution. This closes the loop from "what do I need?" to "the data is ready".

Managing execution of tests: from completed to signed off

Formal completion and genuine test validation are not the same thing; that gap is invisible in execution statistics and only surfaces at go-live. The next agent aims to close it: monitoring execution quality continuously, verifying that Test Steps were documented rather than clicked through, detecting anomalies such as suspiciously high pass rates relative to known system issues, and evaluating whether reported *Defects* carry enough detail for proper resolution. When the checks pass, it will produce a structured test report and a full *Scope Item* → *Test Case* → result traceability matrix, ready for the Test Manager's approval and sign-off.

7.3 – Key Takeaways

1. **This is not a separate tool; it binds into Cloud ALM.** Everything generated lives in Cloud ALM Test Management structures: *Test Plans*, *Test Cases* with tabs for content, relations, and references; notifications; traceability links. The agents work within and strengthen the existing SAP Cloud ALM architecture, not alongside it. Your test team's existing workflows, reports, and integrations remain intact.

2. **AI enhances what is already there.** The agents worked because the content was in the system: linked *Requirements*, *User Stories* with Acceptance Criteria, a process model. AI leverages existing structured content; it does not create quality from nothing. Investing in good process documentation and structured *Requirements* pays off twice.

3. **Human judgment is the irreplaceable layer.** Every agent in the pipeline includes a human review gate. The agents produced the first draft; you provided the quality check. Catching a role mismatch, a missing precondition, a *Scope Item* that does not belong: that is the test expert's contribution, and it matters as much as ever.

4. **Traceability is what makes it trustworthy.** The chain runs in two directions at once. Every *Test Case* links to the *Test Plan* and the scope; every *Scope Item* in that scope links to the process, the *User Stories*, and the Acceptance Criteria that defined it. These two structures meet at scope, which means a test result traces all the way back to a specific process path and the acceptance criterion it was designed to prove. When a test passes, you know what passed, what it means for the process, and which criterion it covers. That chain is what allows you to demonstrate testing assurance to auditors, project managers, and stakeholders with confidence.

Questions & Discussion

Some questions to take into the discussion:

1. Where in your current testing process would agents like these save the most time?
2. What inputs do you have today that you would want to feed to a scoping agent? (*Requirements* in Jira? Process models in Signavio? Interview recordings?)
3. At which stage would your team want to review and confirm before the agent continues? Where does the human gate matter most?

Feedback

We would love to hear what you think. You will find us after the session, or reach us through the session feedback channels at the event.

Thank You

Thank you for your time, your curiosity, and your engagement today.

TM261 – Agent-Led Test Management: Hands-on Lab with Joule in SAP Cloud ALM

This material remains available after the session at the SAP TechEd GitHub repository.